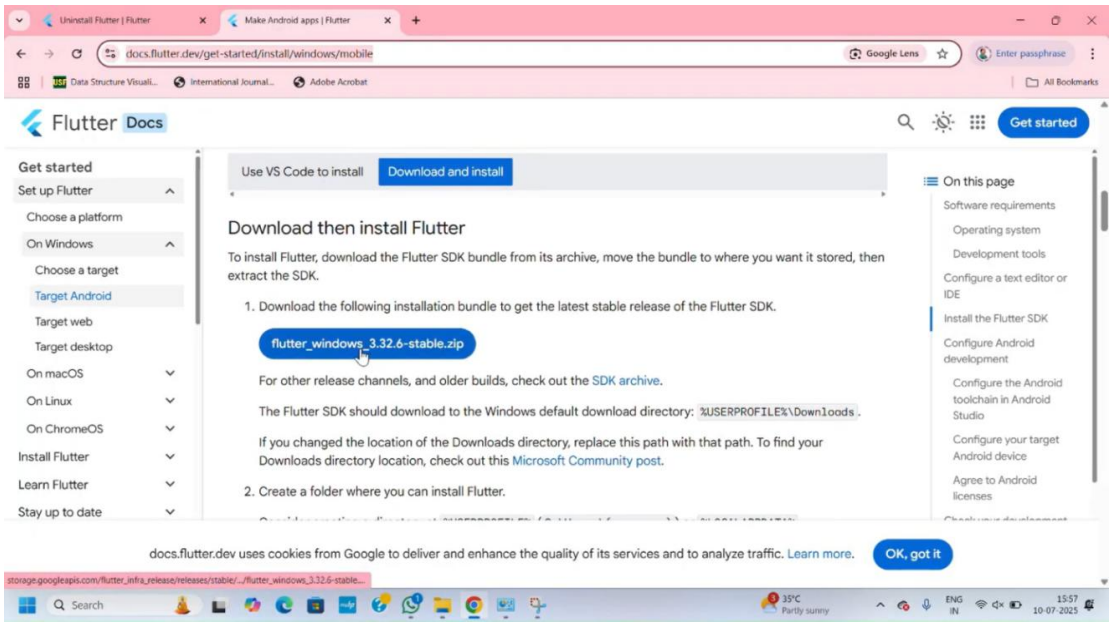
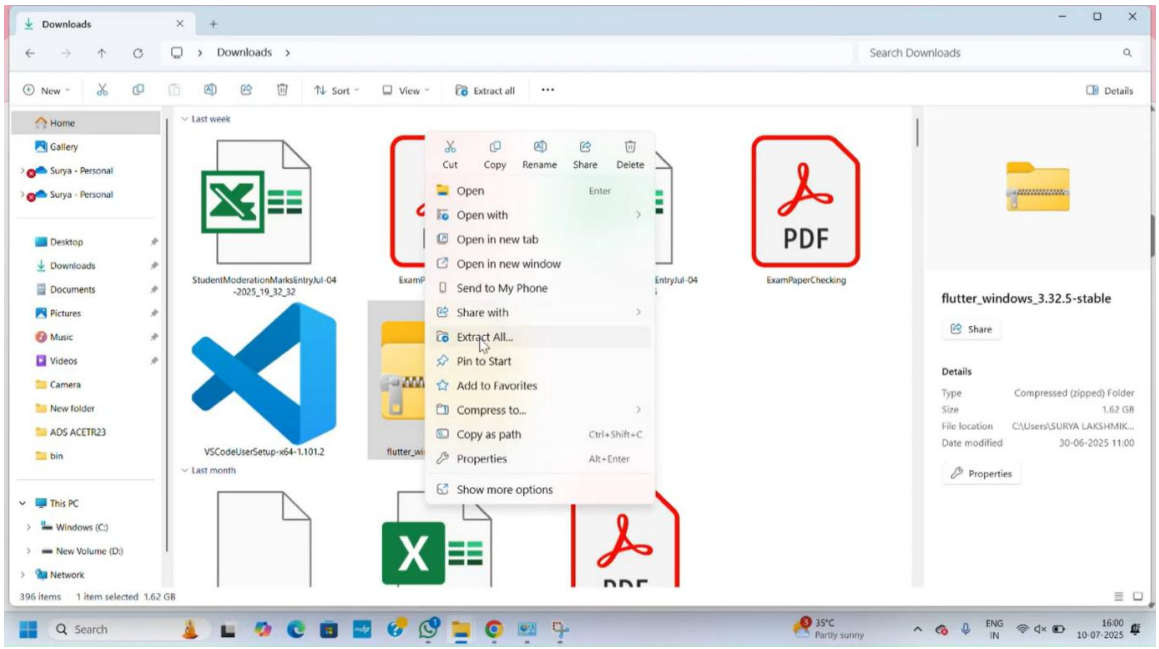
1.) Install Flutter and Dart SDK.

STEP-1: Download flutter SDK for windows OS from

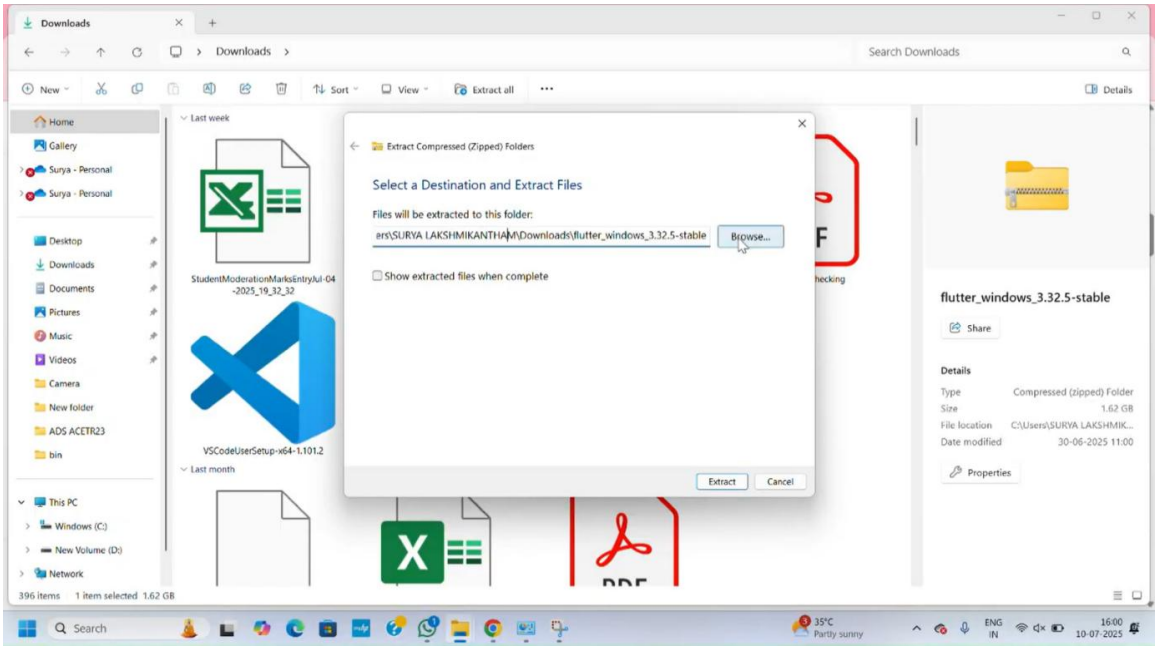
[https://docs.flutter.dev/install/manual] URL.



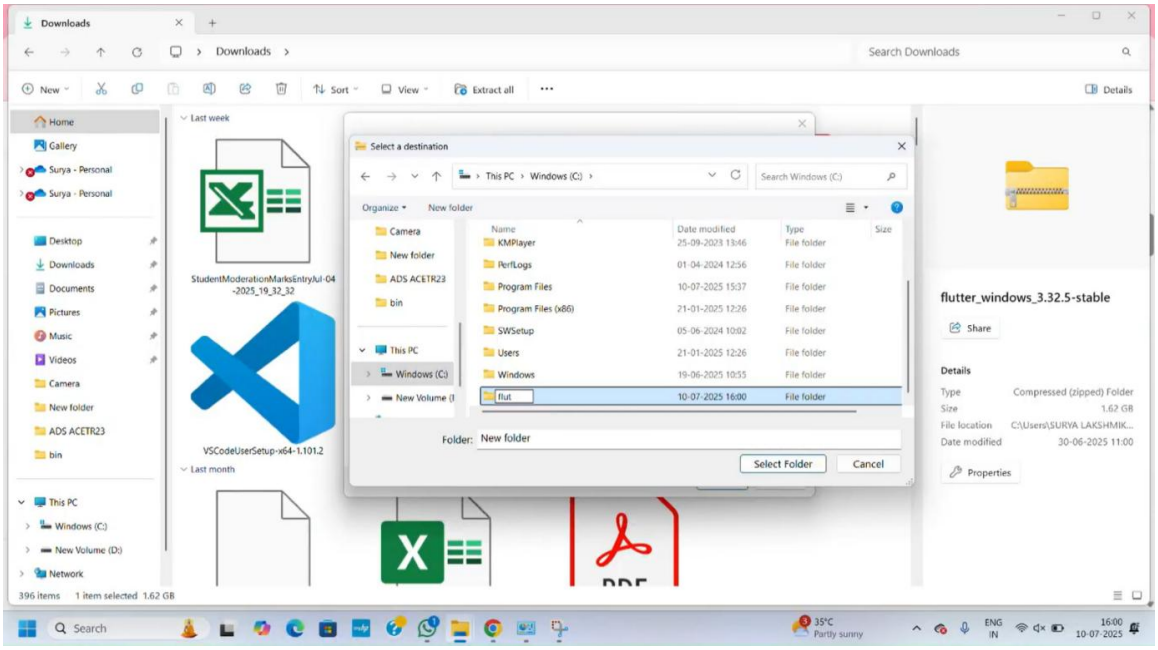
STEP-2: Extraction of flutter SDK into a specific folder.



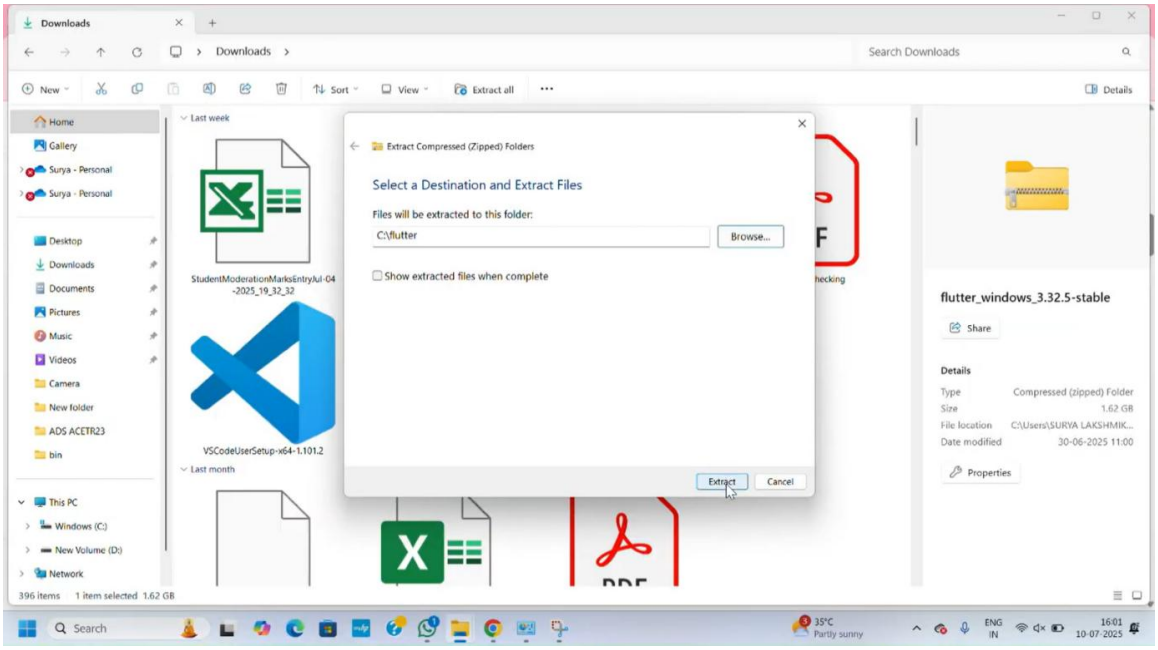
STEP-3: Once it is downloaded, extraction must be done. Click on the Zip file to extract..



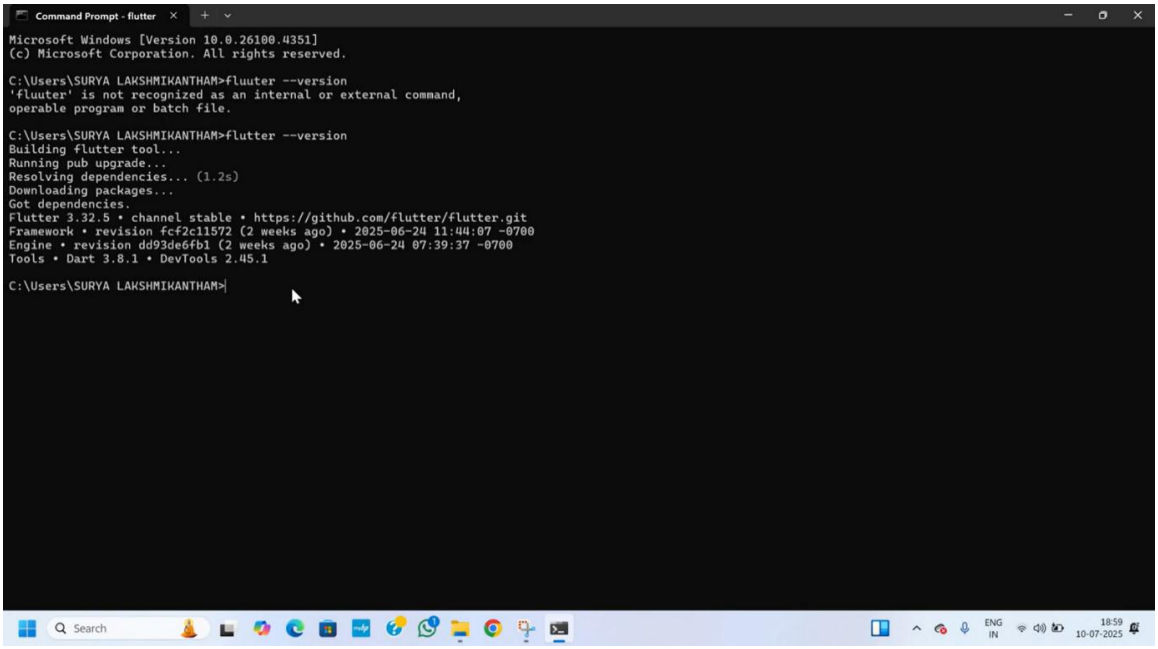
STEP-4: we have to select the folder



STEP-5: Once, the folder is created, extract the contents, by selecting the folder.



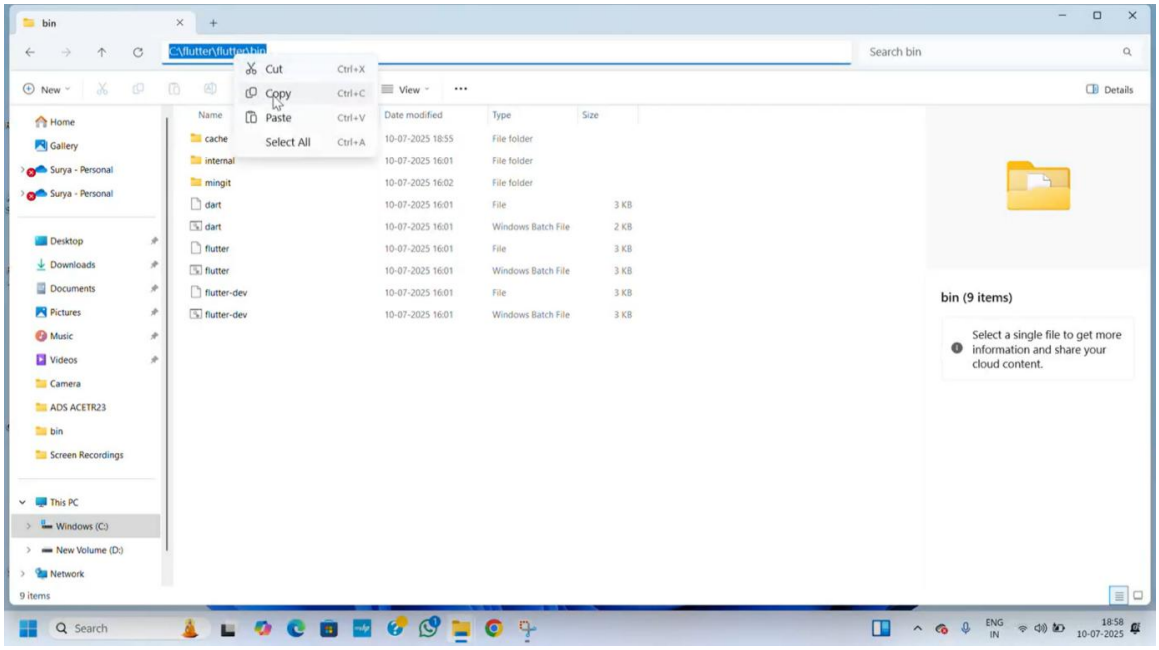
STEP-6: to check whether the flutter SDK is successfully installed, go to command prompt and type **flutter –version** and click enter



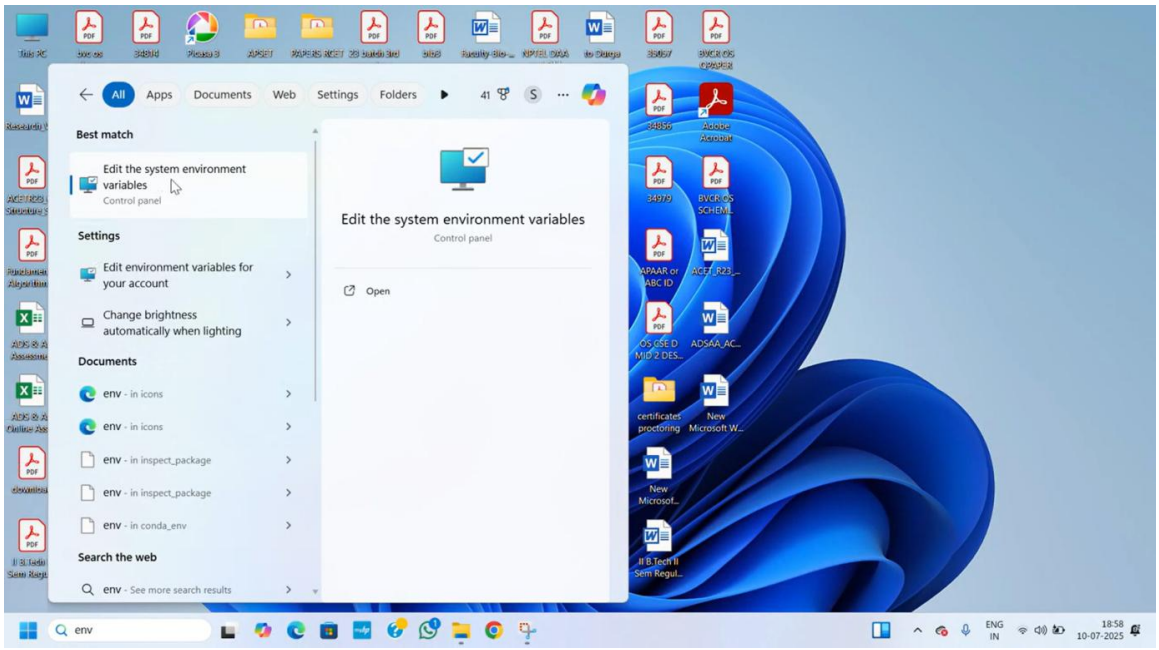
Setting Environmental Variable

STEP-1: To set up the path, go to folder created on c-drive and then move to the flutter and then to the bin.

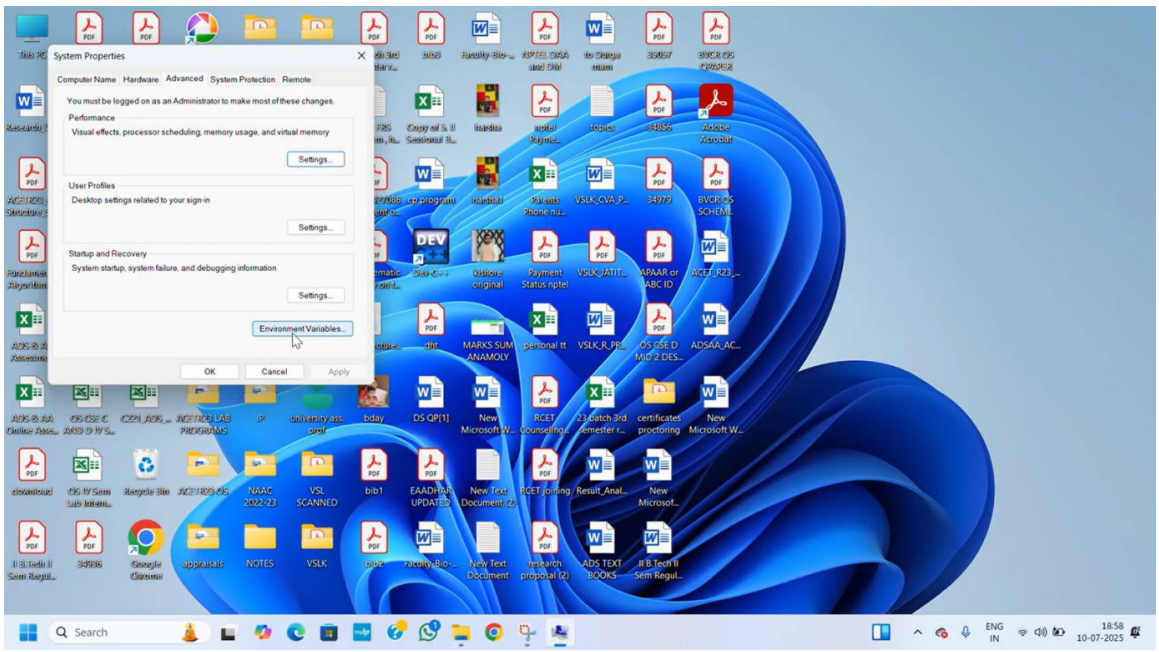
Now copy the path **c:\FLUTTER\flutter\bin**



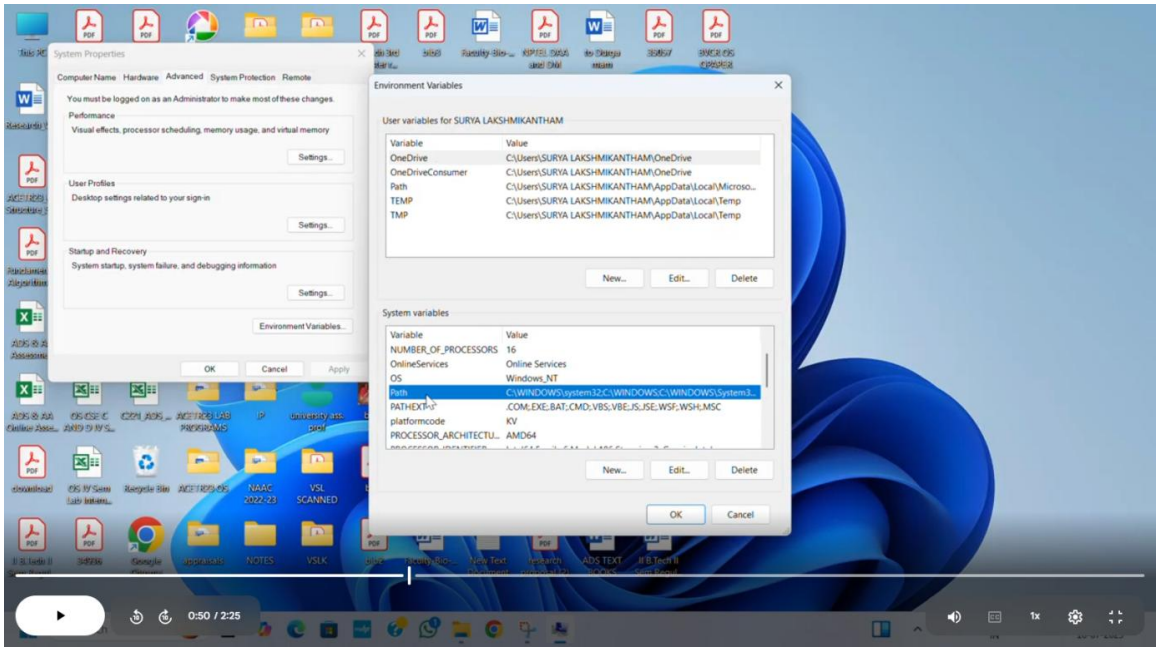
STEP-2: Then go to "Edit the system Environment Variables" control panel



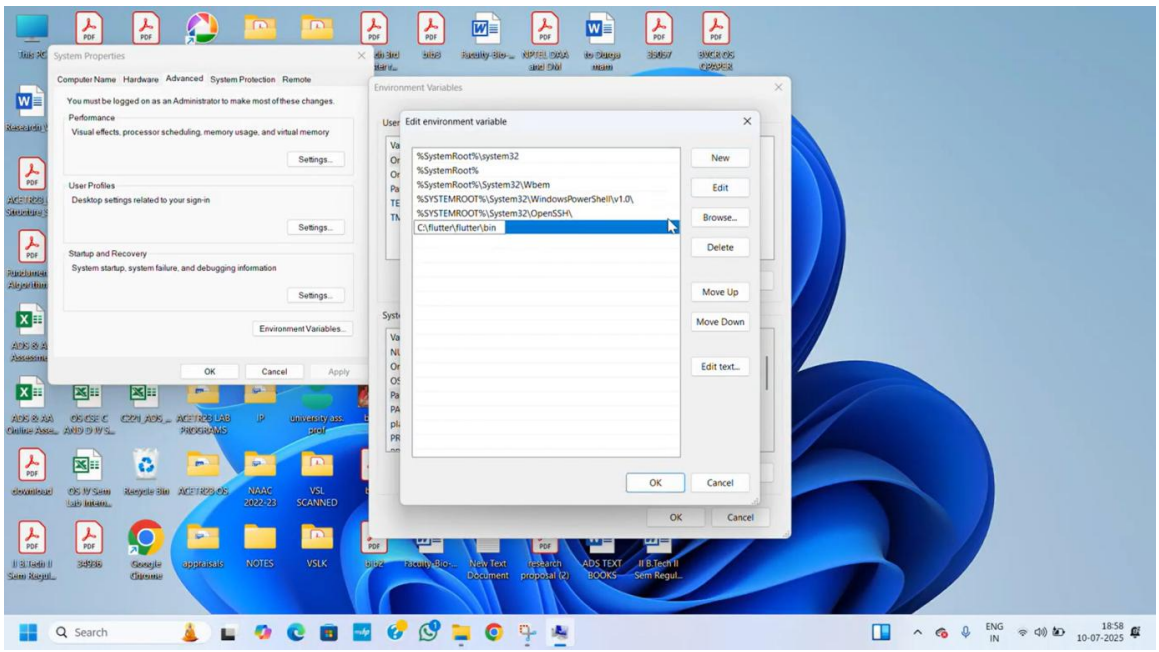
STEP-3: Then click on Environment Variables.



STEP-4: Then move to System variables. Double click on <u>Path</u> and there copy the path link.



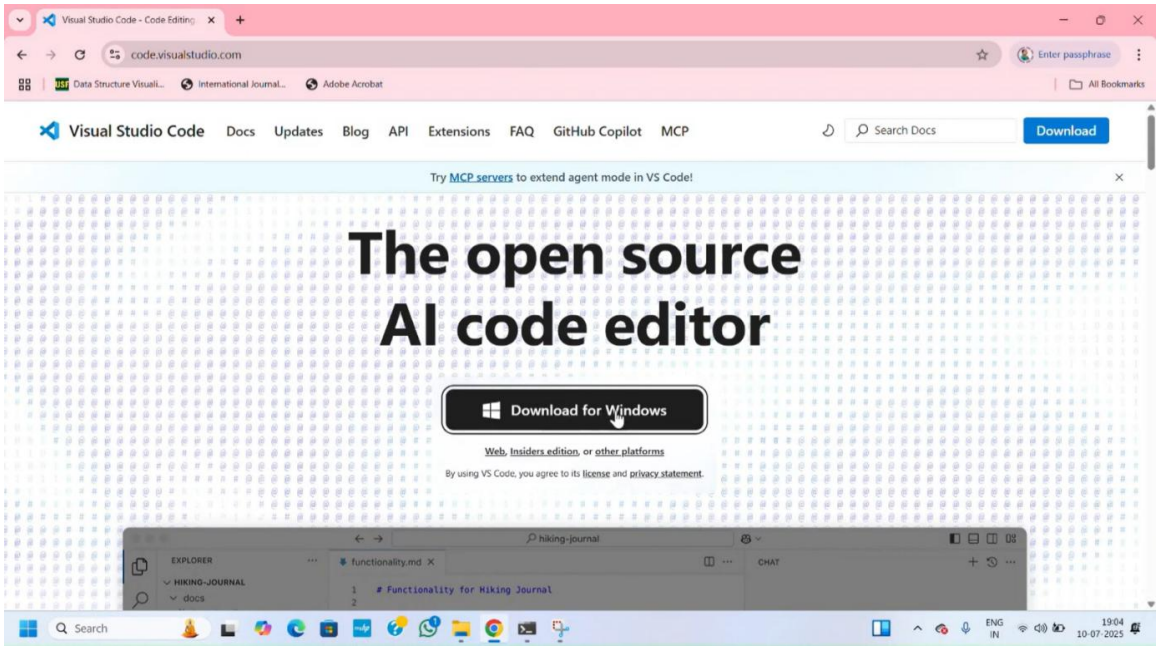
STEP-5: Click on new, then copy the path link.



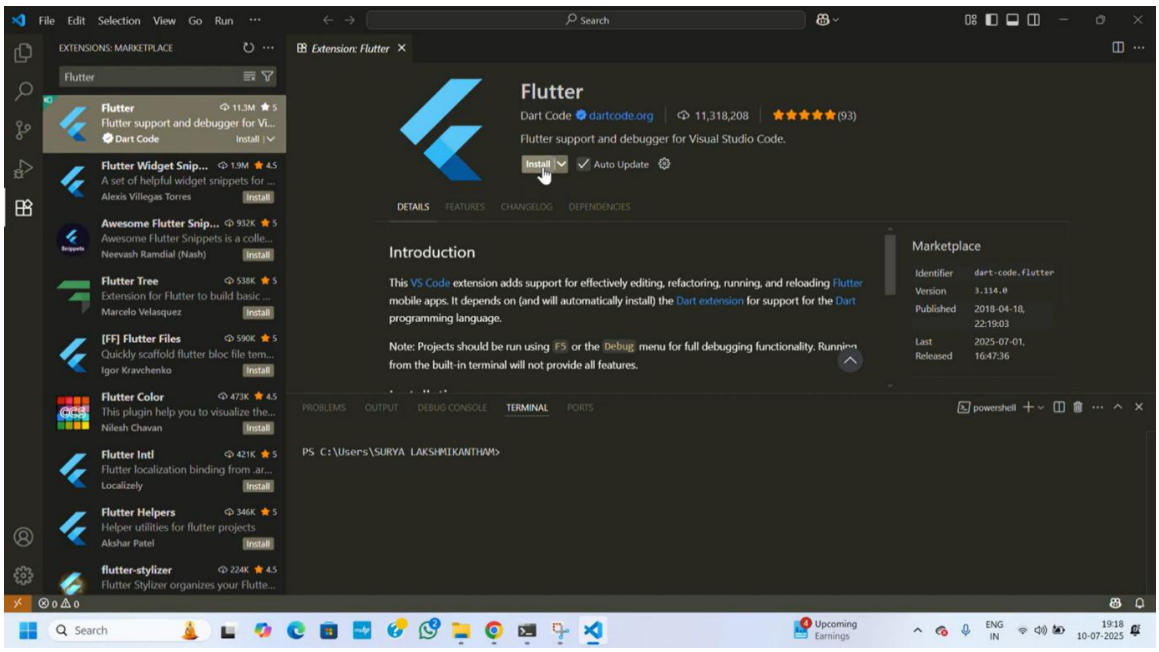
STEP-6: Come back by clicking OK everywhere.

Downloading Flutter and Dart Extensions

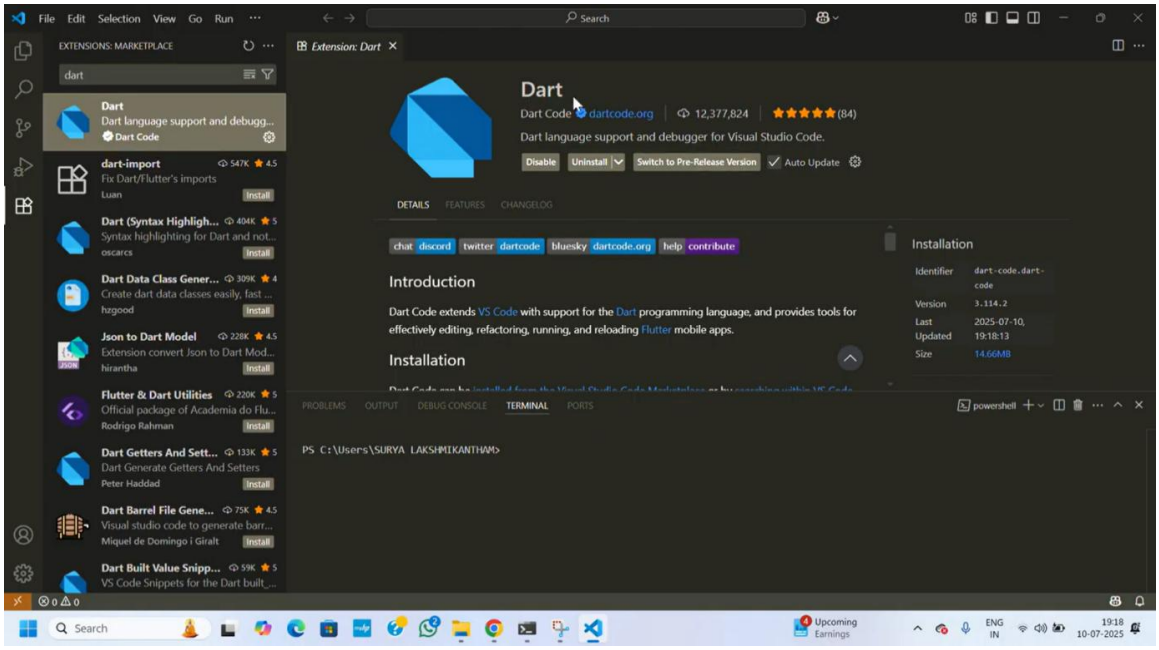
STEP-1: First install the VISUAL STUDIO CODE from user search engine.



STEP-2: Once it is installed, search for the Flutter extension and install it.



STEP-3: Search Dart extension and install it.



1b)

Write a dart program to illustrate basic data types and print function.

Code:

```
void main() {
// Integer variable
 int count = 9;
// Boolean variable
bool flag = true;
// Double (floating point) variable
double marks = 55.78;
// String variable
String name = 'Amar';
// var automatically takes String type
  var hai = "Good Morning";
// Printing using string interpolation
 print("$hai! My name is $name with aggregate $marks");
print("flag is $flag and count is $count");
}
```

OUTPUT:

```
Good Morning! My name is Amar with aggregate 55.78
flag is true and count is 9
```

Write a dart program to read multiple variables of different data types from keyboard and print them.

Code:

```
import 'dart:io';
void main() {
  print('Enter age, height, and name :');
  // Take a line of input, split by spaces, store as List<String>
  List<String> inputs = stdin.readLineSync()!.split(' ');
  // Convert to proper data types
  int age = int.parse(inputs[0]);       // First input → int
  double height = double.parse(inputs[1]);   // Second input → double
  String name = inputs[2];              // Third input → String

  // Print values
  print('Age: $age, Height: $height, Name: $name');
}
```

OUTPUT

```
Enter age, height, and name :
30 180 Ram
Age:30, Height:180.0, Name:Ram
```

Write a dart program on logical operators

code:

```
void main() {
  bool a = true;
  bool b = false;
  // logical AND (&&)
  print('a && b = ${a && b}');
  // logical OR (||)
  print('a || b = ${a || b}');
  // logical NOT (!)
  print('!a = ${!a}');
  print('!b = ${!b}');
  // combining expressions
  int x = 10;
  int y = 20;
  print('(x > 5) && (y > 30) = ${(x > 5) && (y > 30)}');
  print('(x > 5) && (y < 30) = ${(x > 5) && (y < 30)}');
  print('(x < 5) && (y < 30) = ${(x < 5) && (y < 30)}');
  print('(x < 5) || (y > 15) = ${(x < 5) || (y > 15)}');
}
```

OUTPUT:

```
a && b = false
a || b = true
!a = false
!b = true
(x>5) && (y>30) = false
(x>5) && (y<30) = true
(x<5) && (y<30) = false
(x<5) || (y>15) = true
```

Write a Dart program to read number from user, and find whether it is negative or positive. If positive , check whether it is even or odd.

PROGRAM :

```
import 'dart:io';

void main() {
 print("Enter a number:");
 int number = int.parse(stdin.readLineSync()!);
 // Simple if statement
 if (number > 0) {
  print("The number is positive.");
  if (number % 2 == 0) {
   print("The number is even.");
  } else {
   print("The number is odd.");
  }
 } else if (number < 0) {
  print("Negative number");
 } else {
  print("Zero");
 }
}
```

OUTPUT :

```
PS D:\CSE C FLUTTER\dart_application_3\bin> dart .\dart_application_3.dart
Enter a number:
9
The number is positive.
The number is odd.
PS D:\CSE C FLUTTER\dart_application_3\bin> dart .\dart_application_3.dart
Enter a number:
0
Zero
```

WRITE A DART PROGRAM TO SIMULATE SIMPLE CALCULATOR

```
import 'dart:io';

void main() {

/ //Simple Calculator
  print("\n--- Simple Calculator ---");
  print("Enter first number:");
  double num1 = double.parse(stdin.readLineSync()!);

  print("Enter operator (+, -, *, /):");
  String op = stdin.readLineSync()!;

  print("Enter second number:");
  double num2 = double.parse(stdin.readLineSync()!);

  double result;

  switch (op) {
    case '+':
      result = num1 + num2;
      print("Result: $result");
      break;
    case '-':
      result = num1 - num2;
      print("Result: $result");
      break;
    case '*':
      result = num1 * num2;
      print("Result: $result");
      break;
    case '/':
      if (num2 != 0) {
        result = num1 / num2;
        print("Result: $result");
      } else {
        print("Error: Division by zero");
      }
      break;
    default:
      print("Invalid operator");
  }
}
```

OUTPUT :

--- Simple Calculator ---

Enter first number:

5

Enter operator (+, -, *, /):

+

Enter second number:

3

Result: 8.0

2 a) Explore various Flutter widgets (Text, Image, Container, etc.).

Creating Flutter Widget Text :

```
import 'package:flutter/material.dart';
void main() {
 runApp(MyApp());
}
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text("This is AppBar"),
        titleTextStyle: TextStyle(
          fontSize: 45,
          fontWeight: FontWeight.w600,
          color: Colors.pink,
        ),
        backgroundColor: Colors.amberAccent,
      ),
    ),
  );
 }
}
```

- runApp(MyApp()): Launches your app.
- MaterialApp: Wraps your app in Material Design.
- Scaffold: Provides basic layout like appBar, body.
- AppBar: Top bar with title.
- Text: Displays a string on screen.
- Center: Centers the text on the screen.

OUTPUT :



Creating Flutter Widget Image:

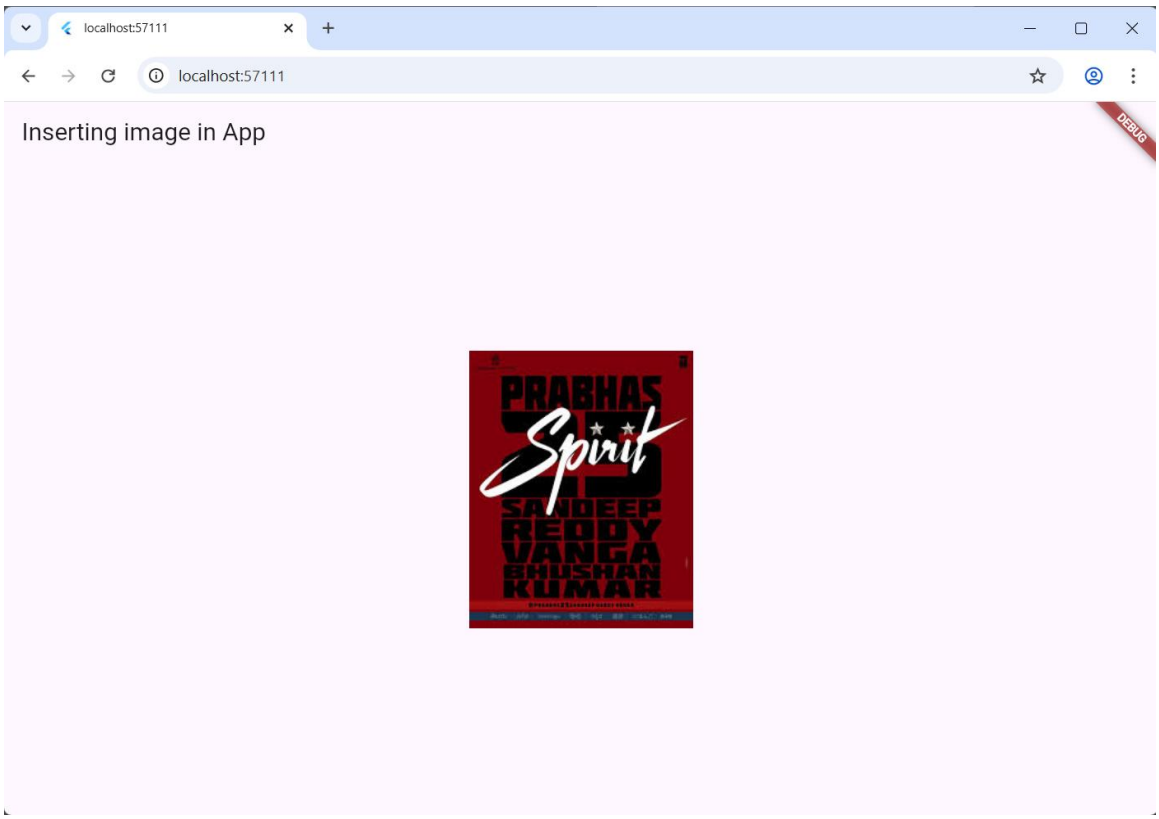
```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text("Inserting image in App")),
        body: Center(child: Image.asset('assets/vangod_spirit.jpeg')),
      ),
    );
  }
}
```

Steps to Add Local Image:

1. Create a folder in your project with name **assets**
2. Place your image (e.g., vangod_spirit.jpeg) inside that folder.
3. Edit your **pubspec.yaml** file in the project folder :

```
flutter:
  assets:
    - assets/vangod_spirit.jpeg
```

OUTPUT :


localhost:57111
localhost:57111
Inserting image in App
DEBUG
PRABHAS
Spirit
SANDEEP
REDDY
VANGA
BHUSHAN
KUMAR

Creating Flutter Widget Container:

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Container Example')),
        body: Center(
          child: Container(
            width: 150,
            height: 150,
            padding: EdgeInsets.all(50),
            margin: EdgeInsets.all(50),
            decoration: BoxDecoration(
              color: Colors.blue[200],
              borderRadius: BorderRadius.circular(22),
              border: Border.all(color: Colors.blue, width: 12),
            ),
            child: Center(
              child: Container(color: Colors.orange, width: 100, height: 100),
```

```
            ),
          ),
        ),
      ),
    );
  }
}
```

Attribute	Purpose
width	Sets the container's width (in logical pixels)
height	Sets the container's height
color	Background color (use directly only if no decoration is used)
alignment	Aligns the child widget inside the container (e.g., Alignment.center)
padding	Space **inside** the container, around its child
margin	Space **outside** the container, around its border
child	The single widget placed inside the container
decoration	Used for styling like background image, gradient, border, shadows

OUTPUT :



2 b) Implement different layout structures using Row, Column, and Stack widgets.

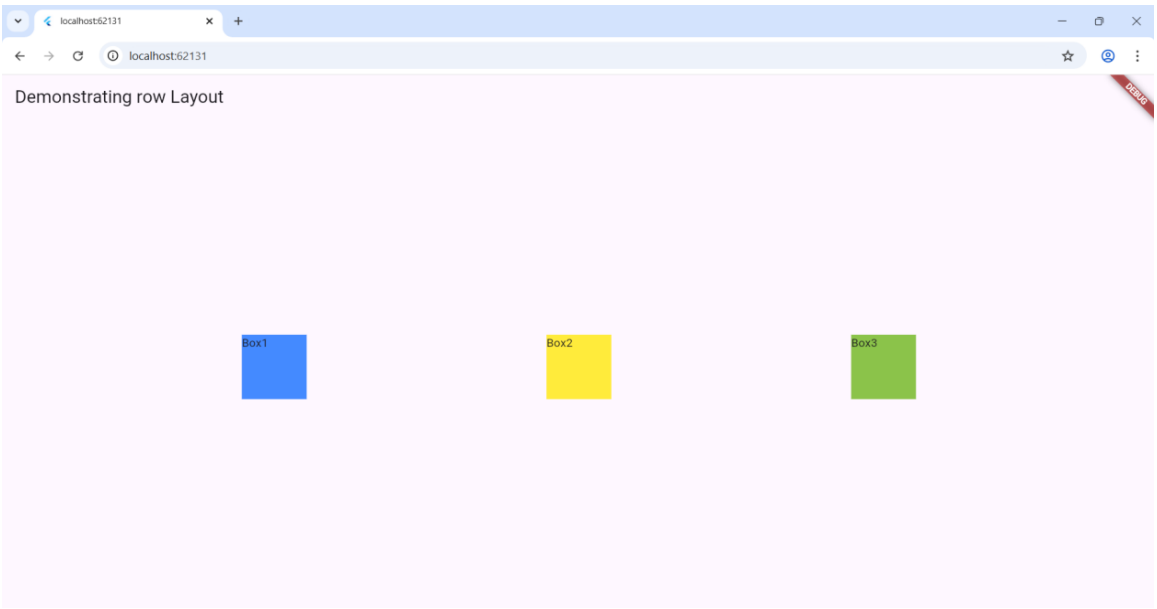
Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';
void main() {
 runApp(MyApp());
}
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   home: Scaffold(
    appBar: AppBar(title: Text('Demonstrating row Layout')),
    body: Center(
     child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
       Container(
        height: 80,
        width: 80,
        color: Colors.blueAccent,
        child: Text("Box1"),
       ),
       Container(
```

```
            height: 80,
            width: 80,
            color: Colors.yellow,
            child: Text("Box2"),
          ),
          Container(
            height: 80,
            width: 80,
            color: Colors.lightGreen,
            child: Text("Box3"),
          ),
        ],
      ),
    ),
  ),
 );
 }
}
```

- Row: Lays out its children **horizontally**
- mainAxisAlignment.spaceEvenly: Distributes the children **with equal spacing**
- crossAxisAlignment.center: Aligns all boxes **vertically in the center**
- Three Containers with different background colors and labels

OUTPUT :


localhost:62131
localhost:62131
Demonstrating row Layout
DEBUG
Box1
Box2
Box3

Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text("Demonstrating Column Layout")),
        body: Column(
          mainAxisAlignment: MainAxisAlignment.start,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Container(
              height: 55,
              width: 55,
              child: Text("Box1"),
              color: Colors.blueAccent,
            ),
            Container(
              height: 55,
              width: 55,
```

```
            child: Text("Box2"),
            color: Colors.pinkAccent,
          ),
          Container(
            height: 55,
            width: 55,
            child: Text("Box3"),
            color: Colors.tealAccent,
          ),
        ],
      ),
    ),
  );
 }
}
```

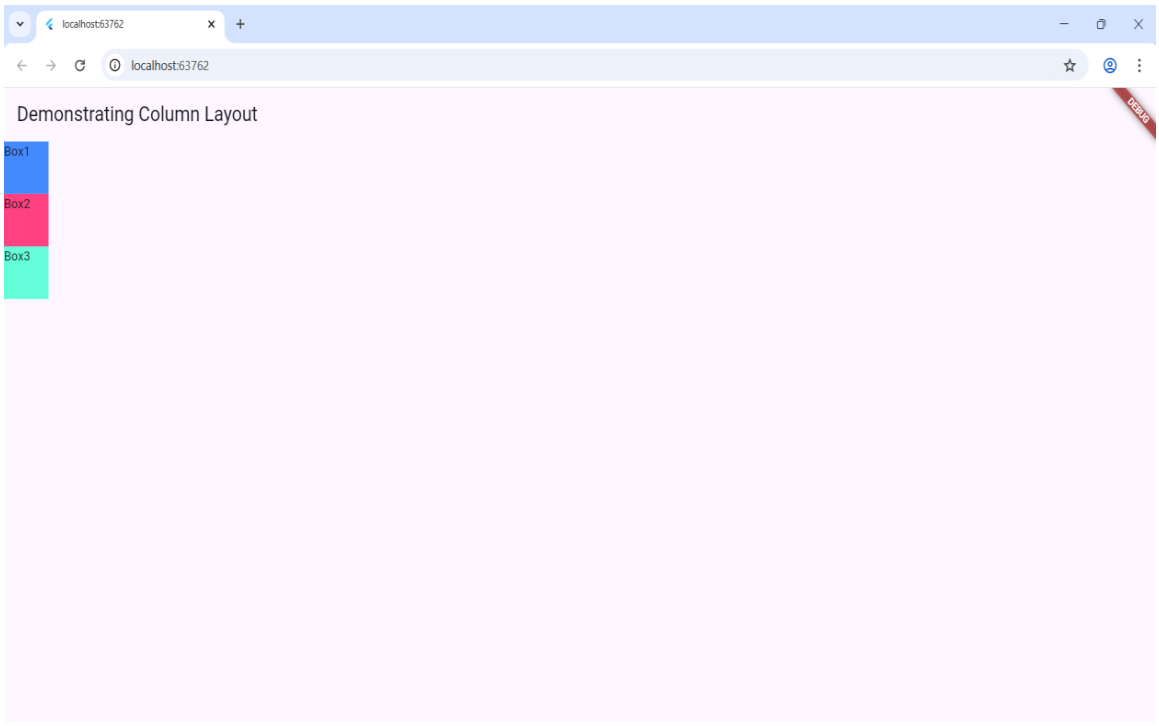
mainAxisAlignment

- **Defines how children are placed along the main axis.**
- For a Row, the main axis is **horizontal**.
- For a Column, the main axis is **vertical**.]

c**rossAxisAlignment**

- **Defines how children are placed along the cross axis.**
- For a Row, the cross axis is **vertical**.
- For a Column, the cross axis is **horizontal**.

OUTPUT :

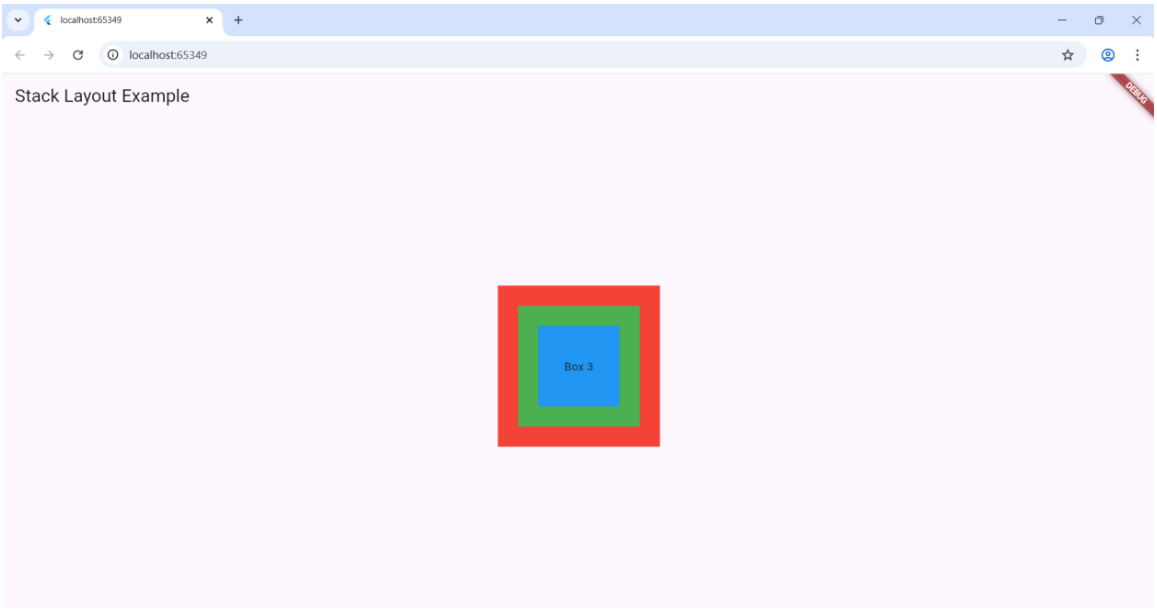

localhost:63762
localhost:63762
Demonstrating Column Layout
DEBUG
Box1
Box2
Box3

Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';
void main()
{
runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Stack Layout Example')),
        body: Center(
          child: Stack(
            alignment: Alignment.center,
            children: [
              Container(
                width: 200,
                height: 200,
                color: Colors.red,
                child: Center(child: Text('Box 1')),
              ),
              Container(
                width: 150,
```

```
            height: 150,
            color: Colors.green,
            child: Center(child: Text('Box 2')),
          ),
          Container(
            width: 100,
            height: 100,
            color: Colors.blue,
            child: Center(child: Text('Box 3')),
          ),
        ],
      ),
    ),
  ),
 );
 }
}
```

OUTPUT :


localhost:65349
localhost:65349
Stack Layout Example
DEBUG
Box 3

3 a) Design a responsive UI that adapts to different screen sizes.

B**eginner-friendly responsive UI design in Flutter** that adapts gracefully to different screen sizes (like mobile and tablet). It uses:

- MediaQuery to detect screen width
- LayoutBuilder to switch layout dynamically
- Flexible for resizing elements
- Clean UI using Containers and Text widgets

PROGRAM :

```
import 'package:flutter/material.dart';

void main()

{

 runApp(MyResponsiveApp());

}

class MyResponsiveApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      title: 'Responsive UI',

      theme: ThemeData(primarySwatch: Colors.blue),

      home: ResponsiveHomePage(),

    );

  }

}

class ResponsiveHomePage extends StatelessWidget {

  @override
```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Responsive UI Example')),
    body: LayoutBuilder(
      builder: (context, constraints) {
        if (constraints.maxWidth < 600) {
          //Small screen: use vertical layout
          return _buildVerticalLayout();
        } else {
          //Large screen: use horizontal layout
          return _buildHorizontalLayout();
        }
      },
    ),
  );
}

//Mobile View
Widget _buildVerticalLayout() {
  return Column(
    children: [
      _buildBox(Colors.red, 'Box 1'),
      SizedBox(height: 10),
      _buildBox(Colors.green, 'Box 2'),
```

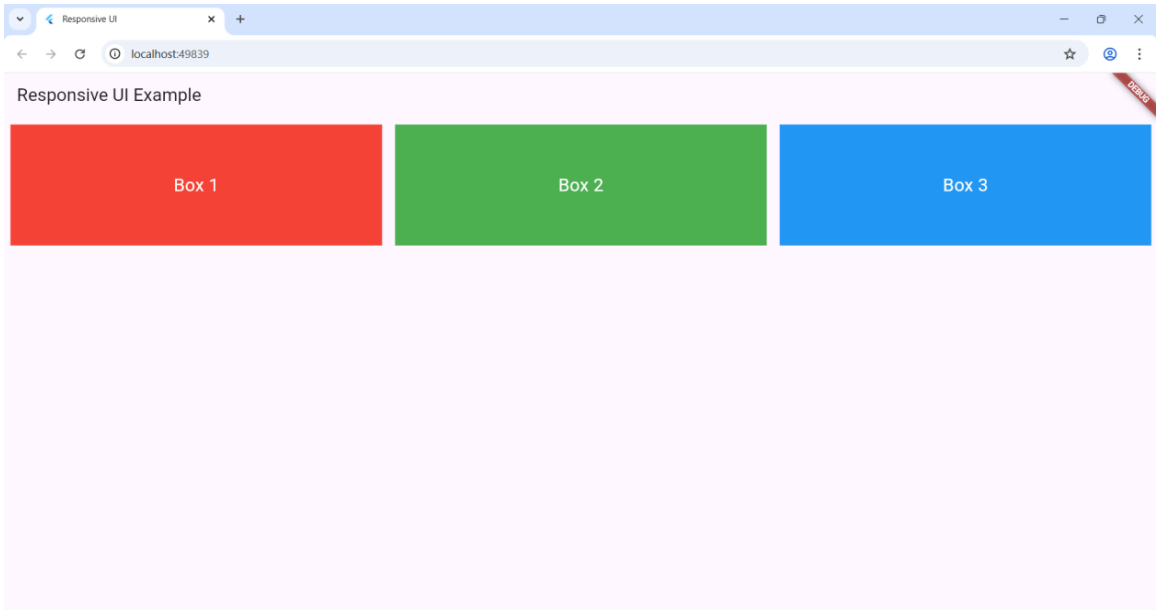
```
      SizedBox(height: 10),
      _buildBox(Colors.blue, 'Box 3'),
    ],
  );
}

// 💻 Tablet/Desktop View
Widget _buildHorizontalLayout() {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      Flexible(child: _buildBox(Colors.red, 'Box 1')),
      Flexible(child: _buildBox(Colors.green, 'Box 2')),
      Flexible(child: _buildBox(Colors.blue, 'Box 3')),
    ],
  );
}

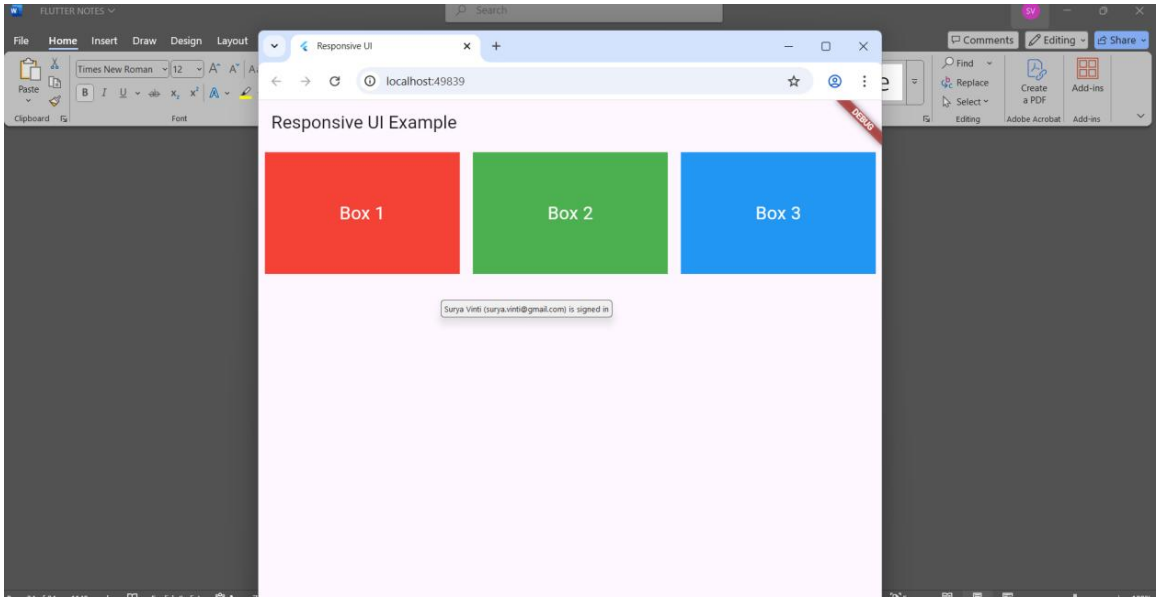
// Reusable UI block
Widget _buildBox(Color color, String label) {
  return Container(
    height: 150,
    margin: EdgeInsets.all(8),
    color: color,
```

```
      child: Center(
        child: Text(
          label,
          style: TextStyle(fontSize: 22, color: Colors.white),
        ),
      ),
    );
  }
}
```

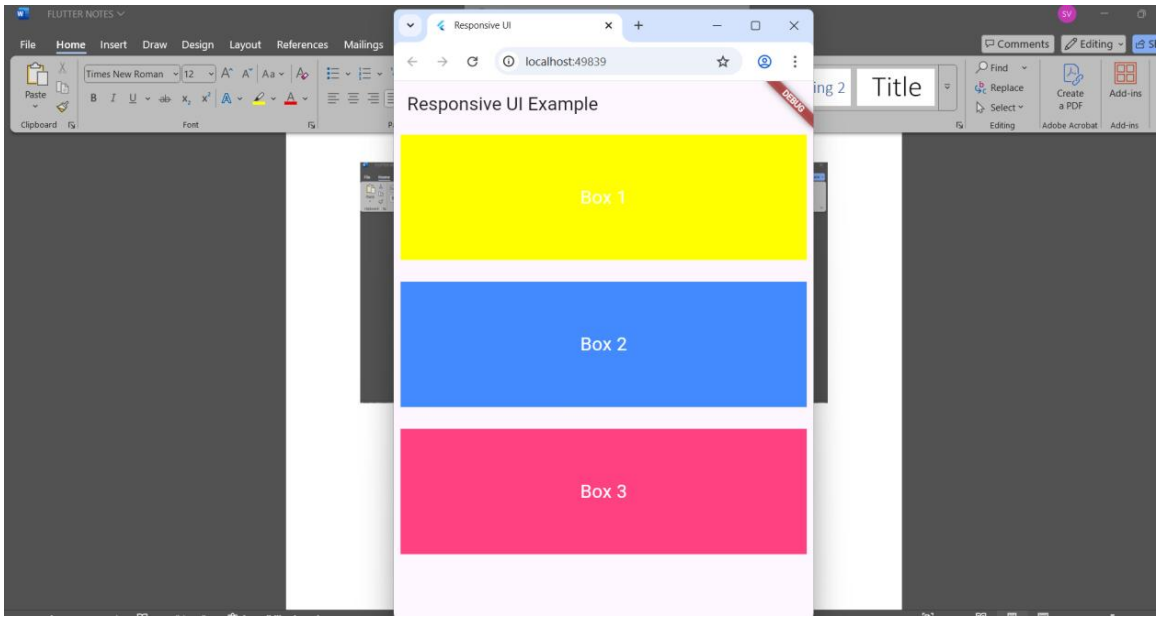
OUTPUT :



DESKTOP/TAB VIEW



DESKTOP/TAB VIEW



MOBILE VIEW

3 b) Implement media queries and breakpoints for responsiveness.

```
import 'package:flutter/material.dart';
void main()
{
runApp(MyResponsiveApp());
}
class MyResponsiveApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Responsive UI with MediaQuery',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: ResponsiveHomePage(),
    );
  }
}

class ResponsiveHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // 📏 Screen width from MediaQuery
    double screenWidth = MediaQuery.of(context).size.width;

    Widget content;

    // 🔁 Check breakpoints
    if (screenWidth < 600) {
      content = _buildMobileLayout();
    } else if (screenWidth >= 600 && screenWidth < 1024) {
      content = _buildTabletLayout();
```

```
  } else {
    content = _buildDesktopLayout();
  }

  return Scaffold(
    appBar: AppBar(title: Text('Responsive Layout')),
    body: content,
  );
}

// 📱 Mobile layout (column)
Widget _buildMobileLayout() {
  return Column(
    children: [
      _buildBox(Colors.red, 'Mobile Box 1'),
      _buildBox(Colors.green, 'Mobile Box 2'),
      _buildBox(Colors.blue, 'Mobile Box 3'),
    ],
  );
}

// 📲 Tablet layout (grid-like)
Widget _buildTabletLayout() {
  return Padding(
    padding: const EdgeInsets.all(8.0),
    child: Wrap(
      spacing: 10,
      runSpacing: 10,
      children: [
        _buildBox(Colors.red, 'Tablet Box 1'),
```

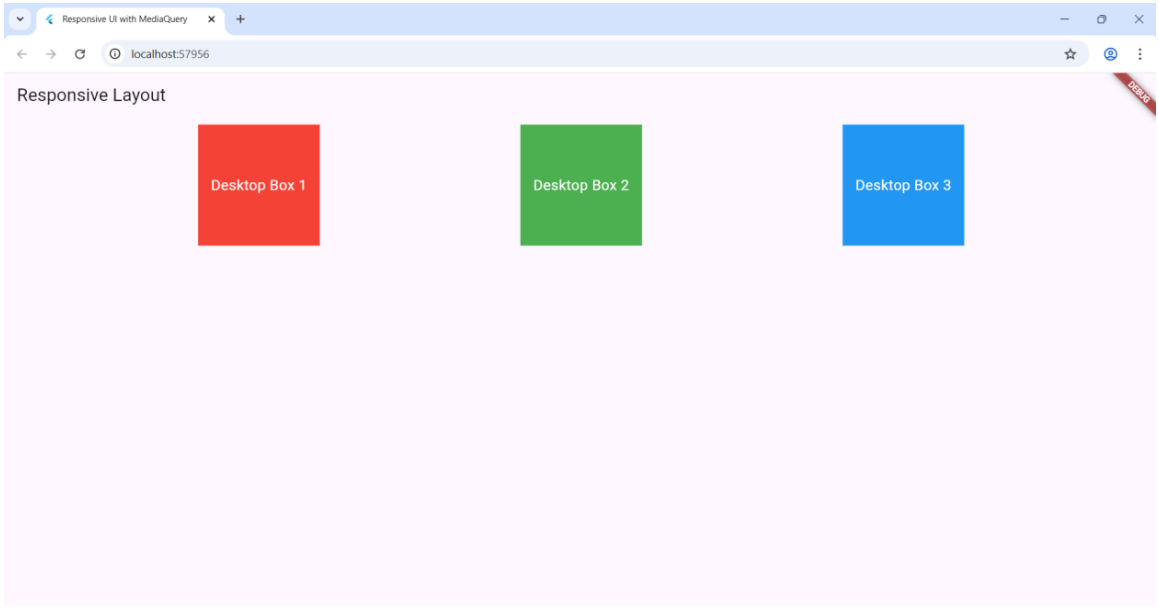
```
        _buildBox(Colors.green, 'Tablet Box 2'),
        _buildBox(Colors.blue, 'Tablet Box 3'),
      ],
    ),
  );
}

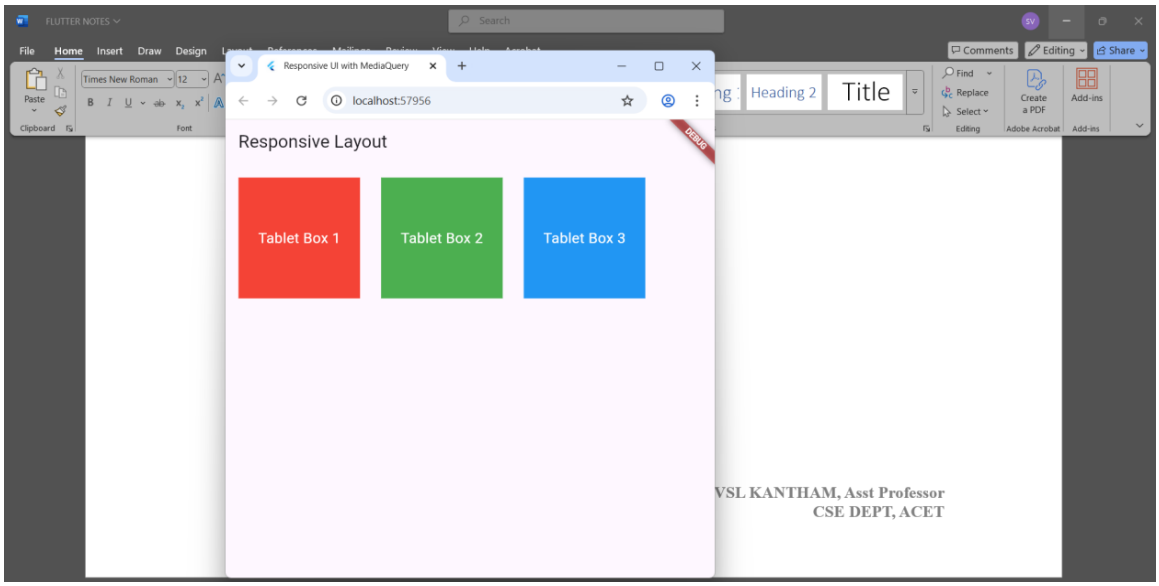
// 💻 Desktop layout (row)
Widget _buildDesktopLayout() {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      _buildBox(Colors.red, 'Desktop Box 1'),
      _buildBox(Colors.green, 'Desktop Box 2'),
      _buildBox(Colors.blue, 'Desktop Box 3'),
    ],
  );
}

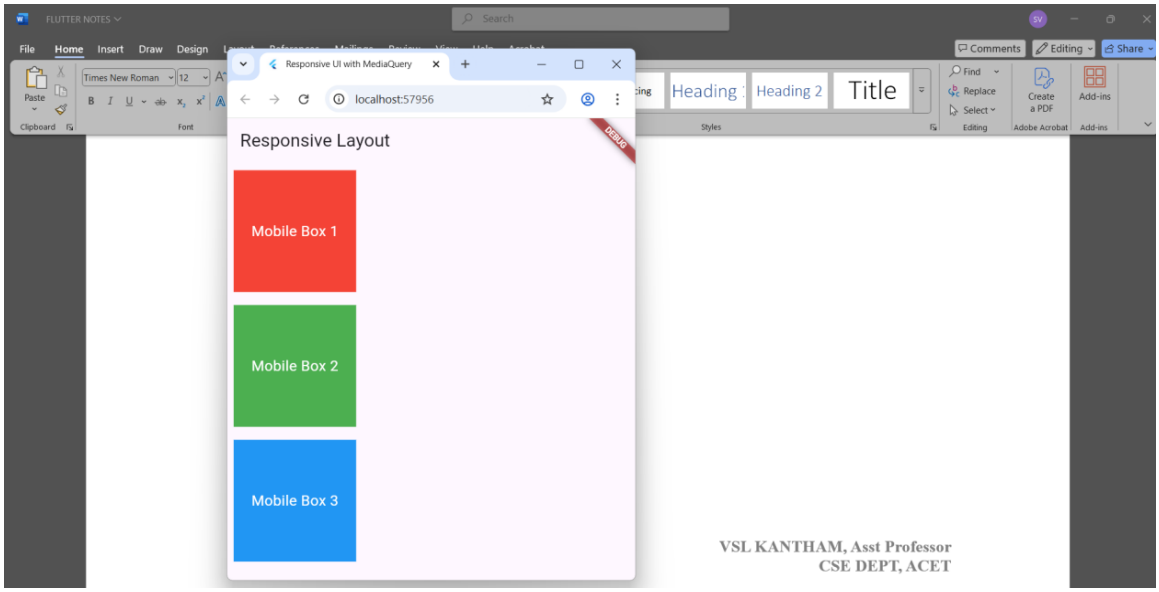
// 🔧 Reusable widget
Widget _buildBox(Color color, String label) {
  return Container(
    width: 150,
    height: 150,
    margin: EdgeInsets.all(8),
    color: color,
    child: Center(
      child: Text(
        label,
        style: TextStyle(color: Colors.white, fontSize: 18),
```

```
        ),
      ),
    );
  }
}
```

OUTPUT:


Responsive UI with MediaQuery
localhost:57956
Responsive Layout
DEBUG
Desktop Box 1
Desktop Box 2
Desktop Box 3


Responsive UI with MediaQuery
localhost:57956
Responsive Layout
DEBUG
Tablet Box 1
Tablet Box 2
Tablet Box 3
Heading 2
Title
VSL KANTHAM, Asst Professor
CSE DEPT, ACET


Responsive UI with MediaQuery
localhost:57956
Responsive Layout
DEBUG
Mobile Box 1
Mobile Box 2
Mobile Box 3
VSL KANTHAM, Asst Professor
CSE DEPT, ACET

4 a) Set up navigation between different screens using Navigator.

- Navigator.push moves to a new screen
- Navigator.pop returns to the previous screen

```
import 'package:flutter/material.dart';

void main() => runApp(MyNavigationApp());

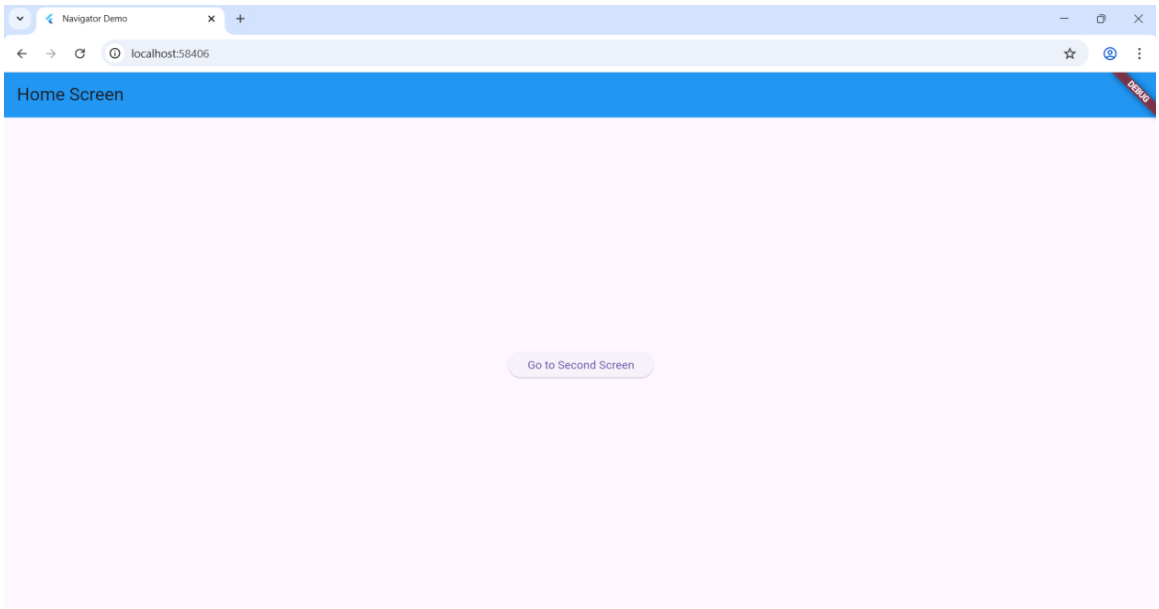
class MyNavigationApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Navigator Demo',
      theme: ThemeData(primarySwatch: Colors.pink),
      home: HomeScreen(),
    );
  }
}

// 🏠 Home Screen
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Home Screen'), backgroundColor: Colors.blue),
      body: Center(
        child: ElevatedButton(
          child: Text('Go to Second Screen'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => SecondScreen()),
            );
          },
        ),
```

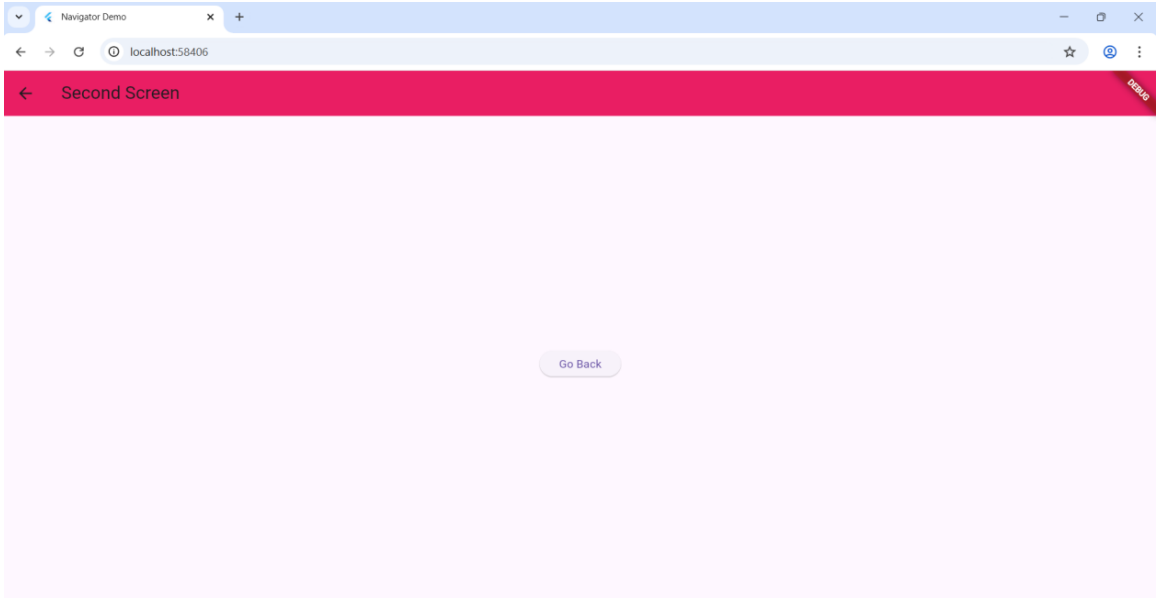
```
      ),
    );
  }
}

// Second Screen
class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Second Screen'),
        backgroundColor: Colors.pink,
      ),
      body: Center(
        child: ElevatedButton(
          child: Text('Go Back'),
          onPressed: () {
            Navigator.pop(context);
          },
        ),
      ),
    );
  }
}
```

OUTPUT:



FIRST SCREEN



SECOND SCREEN

b) Implement navigation with named routes.

```
import 'package:flutter/material.dart';

void main() => runApp(MyNamedRouteApp());

class MyNamedRouteApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Named Route Demo',
      // ✅ Define named routes here
      initialRoute: '/',
      routes: {
        '/': (context) => HomeScreen(),
        '/second': (context) => SecondScreen(),
      },
    );
  }
}

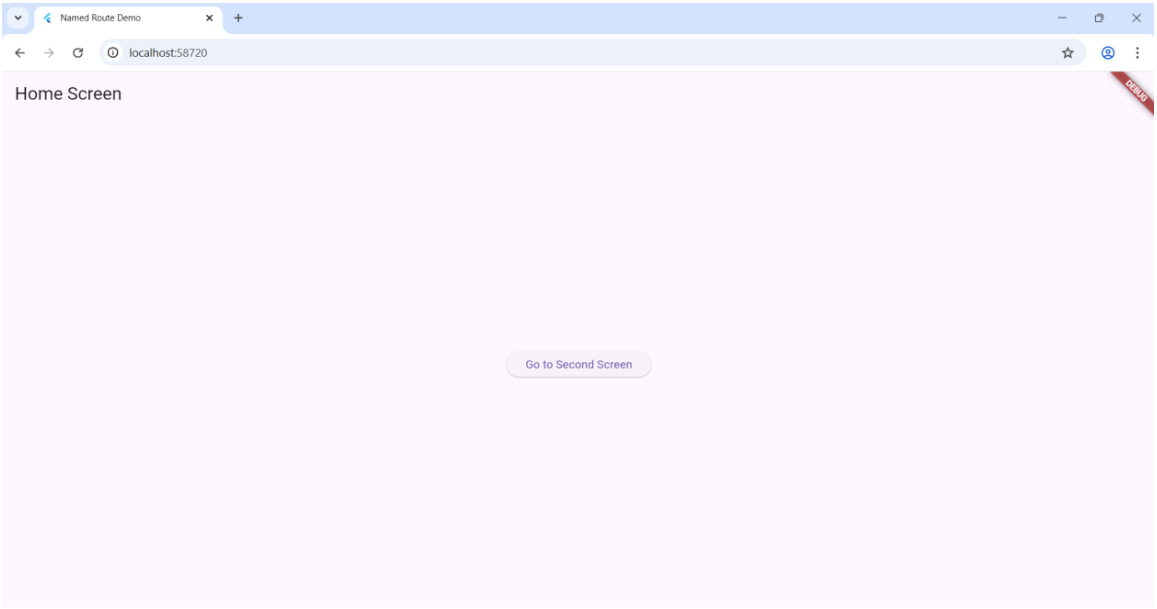
// 🏠 Home Screen
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Home Screen')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go to Second Screen'),
          onPressed: () {
            // 🚀 Navigate using named route
            Navigator.pushNamed(context, '/second');
          },
```

```
        ),
      ),
    );
  }
}

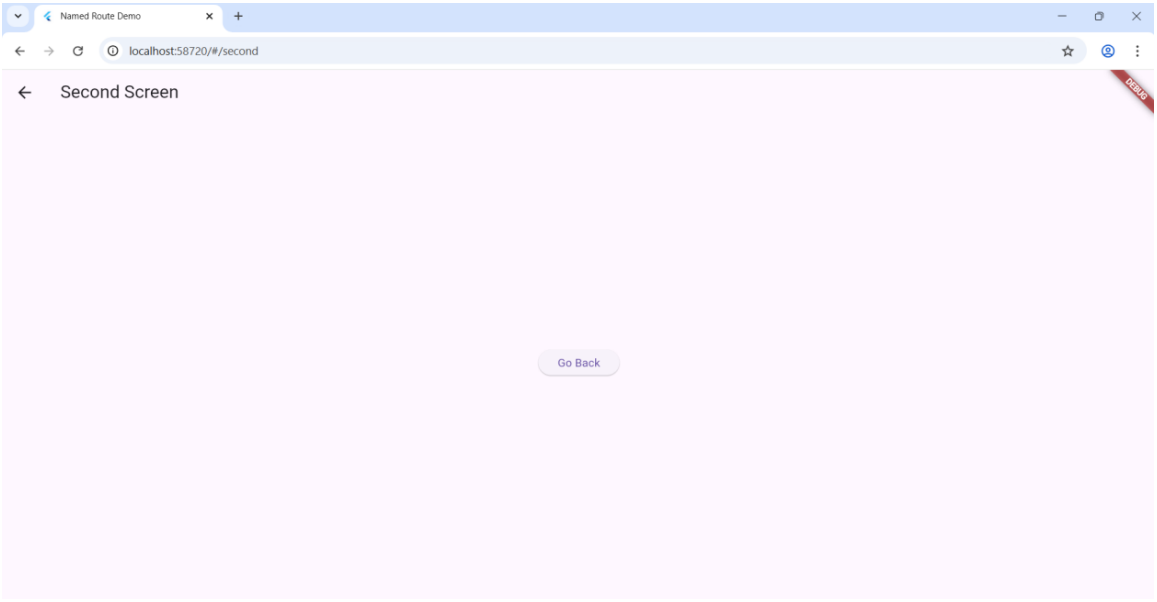
// 📄 Second Screen
class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Second Screen')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go Back'),
          onPressed: () {
            Navigator.pop(context); // ⬅️ Go back
          },
        ),
      ),
    );
  }
}
```

Concept	**Description**
routes	A map of route names to widget builders
initialRoute	The first screen shown when the app starts
Navigator.pushNamed	Navigates using a route name
Navigator.pop	Returns to the previous screen

OUTPUT :



FIRST SCREEN



SECOND SCREEN

5 a) Learn about stateful and stateless widgets.

Stateless Widgets

A Stateless Widget is a widget that does not hold any state. This means that the widget's properties are immutable and cannot change during its lifecycle. Stateless widgets are ideal for displaying static content, such as text or images. This means these widgets are ideal for situations where the UI does not need to change dynamically in response to user interactions or other events.

Characteristics of Stateless Widgets:

- Immutability: Once created, the properties cannot be changed.
- No State Management: They do not manage any internal state.
- Rebuild on Parent Change: They rebuild only when their parent widget changes.

Benefits of Stateless Widgets:

- Simpler to code: Due to their immutable nature, stateless widgets require less code to implement.
- Faster performance: Stateless widgets are rebuilt efficiently, leading to smoother UI rendering.
- Easier to reason about: Their predictable behavior makes them easier to understand and maintain.

Stateful Widgets

A Stateful Widget is a widget that can change its state over time. This means that a Stateful Widget can rebuild itself when its state changes. They are suitable for scenarios where the UI needs to reflect changes based on user interactions, animations, or network responses. A stateful widget consists of two classes: the widget itself and a state class that holds the state. The state class is where you define the mutable state and the logic to manage it.

Characteristics of Stateful Widgets:

- Mutable State: They maintain an internal state that can change during the widget's lifecycle.
- State Management: They manage their state using a separate class that extends state.

- Rebuild on State Change: They rebuild when their internal state changes.

Advantages of Stateful Widgets:

- Dynamic UI: They provide the power to create UIs that respond to user actions and data changes.
- Complex interactions: They are well-suited for handling intricate user interactions and state management.

5b) Implement state management using set State and Provider.

State Management with setState() :

```
import 'package:flutter/material.dart';


void main() {
  runApp(MySetStateApp());
}


class MySetStateApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterScreen(),
    );
  }
}


class CounterScreen extends StatefulWidget {
  @override
  _CounterScreenState createState() => _CounterScreenState();
}


class _CounterScreenState extends State<CounterScreen> {
  int counter = 0;
```

```
  void _incrementCounter() {
    setState(() {
      counter++; // updates UI
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Counter with setState")),
      body: Center(
        child: Text(
          "Counter: $counter",
          style: TextStyle(fontSize: 24),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

OUTPUT:



State Management with Provider:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

// Step 1: Create ChangeNotifier class
class CounterProvider extends ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners(); // tells widgets to rebuild
  }
```

```
}

// Step 2: Wrap app with ChangeNotifierProvider
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterProvider(),
      child: MyProviderApp(),
    ),
  );
}

class MyProviderApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterScreen(),
    );
  }
}

// Step 3: Use Provider in UI
class CounterScreen extends StatelessWidget {
  @override
```

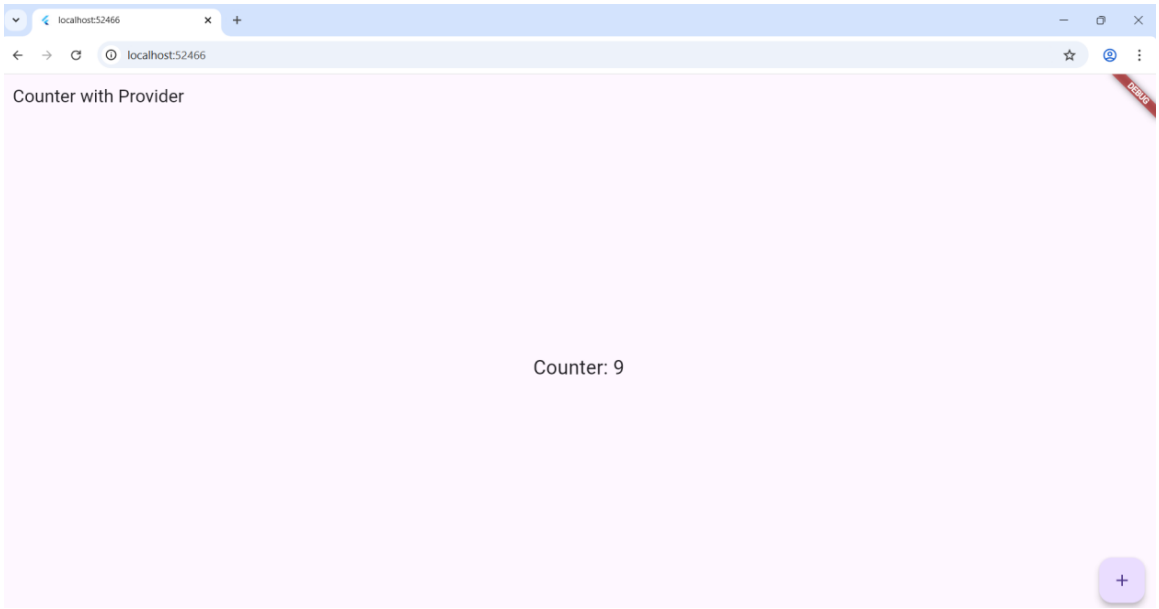
```
  Widget build(BuildContext context) {
    final counterProvider = Provider.of<CounterProvider>(context);

    return Scaffold(
      appBar: AppBar(title: Text("Counter with Provider")),
      body: Center(
        child: Text(
          "Counter: ${counterProvider.counter}",
          style: TextStyle(fontSize: 24),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: counterProvider.increment,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

Add dependency in pubspec.yaml:

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.0.5
```

OUTPUT:


localhost:52466
localhost:52466
Counter with Provider
DEBUG
Counter: 9
+

EXPERIMENT – 6

AIM:
a) Create custom widgets for specific UI elements.
b) Apply styling using themes and custom styles.

Experiment - 6(a)

AIM: Create custom widgets for specific UI elements.

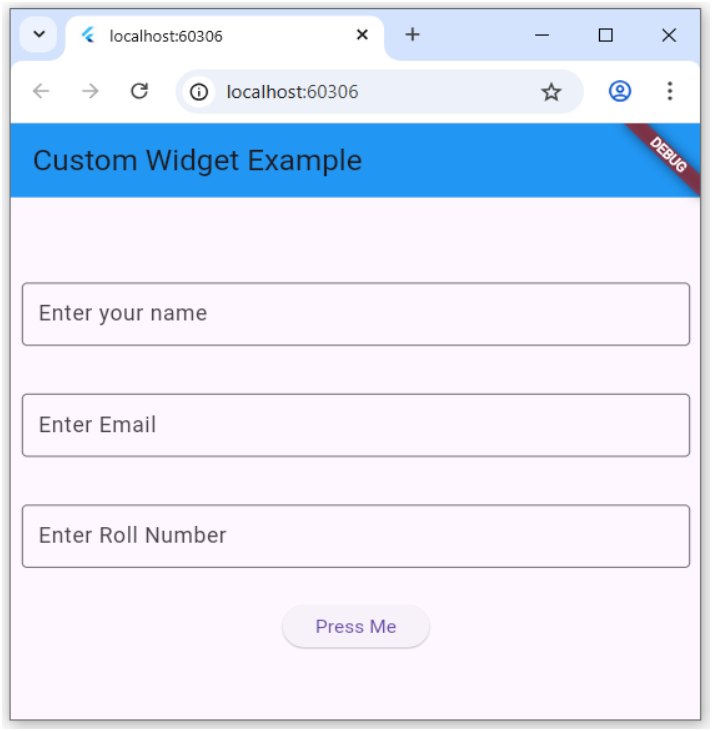
SOLUTION:

```
import 'package:flutter/material.dart';
void main()
{
 runApp(MyApp());
}
class MyApp extends StatelessWidget
{
 @override
 Widget build(BuildContext context)
 {
  return MaterialApp(
   home: Scaffold(
    appBar: AppBar(
    title: Text('Custom Widget Example'),
    backgroundColor: Colors.blue,
    ),
    body: Column(
     mainAxisAlignment: MainAxisAlignment.center, children:
    <Widget>[
     Padding(
      padding: const EdgeInsets.all(8.0),
      child: CustomTextField(
      hintText: 'Enter your name',
      onChanged: (value) { print('Name changed: $value'); },
      ),
     ),
     SizedBox(height: 20),
     Padding(
      padding: const EdgeInsets.all(8.0),
      child: CustomTextField(
       hintText: 'Enter Email',
       onChanged: (value) { print('Name changed: $value');},
      ),
     ),
     SizedBox(height: 20),
     Padding(
      padding: const EdgeInsets.all(8.0),
```

```
            child: CustomTextField(
              hintText: 'Enter Roll Number',
              onChanged: (value) { print('Name changed: $value');},
            ),
          ),
          SizedBox(height: 20),
          CustomButton(
            text: 'Press Me',
            onPressed: () { print('Button pressed!'); },
          ),
        ],
        ),
      ),
    );
  }
}
class CustomButton extends StatelessWidget
{
  final String? text;
  final VoidCallback? onPressed;
  const CustomButton({ Key? key, required this.text, required this.onPressed,
  }) : super(key: key);
  @override
  Widget build(BuildContext context)
  {
    return ElevatedButton(
      onPressed: onPressed,
      child: Text(text!),
    );
  }
}
class CustomTextField extends StatelessWidget
{
  final String hintText;
  final ValueChanged<String> onChanged;
  const CustomTextField({ Key? key, required this.hintText,
    required this.onChanged,}) : super(key: key);
  @override
  Widget build(BuildContext context)
  {
    return TextField(
      onChanged: onChanged,
      decoration: InputDecoration(
        hintText: hintText,
        border: OutlineInputBorder(),
      ),
```

```
    );
  }
}
```

OUTPUT:


localhost:60306
localhost:60306
Custom Widget Example
DEBUG
Enter your name
Enter Email
Enter Roll Number
Press Me

Experiment - 6(b)

AIM: Apply styling using themes and custom styles.

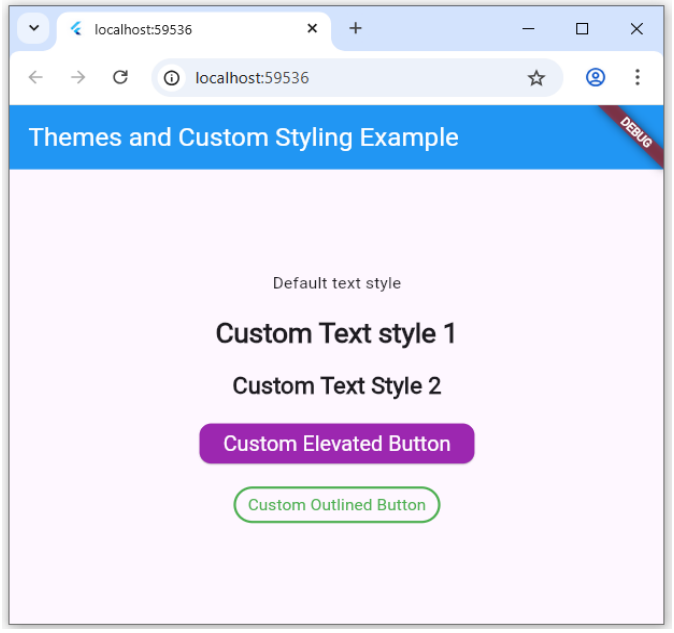
DESCRIPTION:
In Flutter, you can apply styling to your widgets using themes and custom styles to maintain consistency and make your UI more visually appealing.

SOLUTION:

```
import 'package:flutter/material.dart';
void main()
{ runApp(MyApp());
}
class MyApp extends StatelessWidget
{ @override
Widget build(BuildContext context)
{ return MaterialApp(
  theme: ThemeData(
   // Define the overall theme of the app
    primaryColor: Colors.blue,
    fontFamily: 'Roboto',
       // Text Theme
    textTheme: TextTheme(
      displayLarge: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
      headlineMedium: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
      bodyLarge: TextStyle(fontSize: 16),
    ),
        // Elevated Button Theme
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.purple,
        foregroundColor: Colors.white,
        textStyle: TextStyle(fontSize: 18),
        padding: EdgeInsets.symmetric(horizontal: 20, vertical: 15),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(10),
        ),
      ),
    ),
       // Outlined Button Theme
    outlinedButtonTheme: OutlinedButtonThemeData(
      style: OutlinedButton.styleFrom(
        foregroundColor: Colors.green, // Text/Icon color
        side: const BorderSide(color: Colors.green, width: 2), // Border
        padding: const EdgeInsets.all(12),
      ),
    ),
  ),
  home: HomePage(),
);
  }
}
class HomePage extends StatelessWidget {
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Themes and Custom Styling Example'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children:
          <Widget>[
            Text('Default text style'),
            SizedBox(height: 20),
            Text('Text style 1', style: Theme.of(context).textTheme.displayLarge),
            SizedBox(height: 20),
            Text('Text Style 2', style: Theme.of(context).textTheme.headlineMedium),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {},
              child: Text('Custom Elevated Button'),
            ),
            SizedBox(height: 20),
            OutlinedButton(
              onPressed: () {},
              child: const Text('Custom Outlined Button'),
            ),
          ],
        ),
      ),
    );
  }
}
```

OUTPUT:



EXPERIMENT – 7

AIM:
a) Design a form with various input fields.
b) Implement form validation and error handling.

Experiment - 7(a)

AIM: Design a form with various input fields.

DESCRIPTION:
Design a form with various input fields such as text fields, checkboxes, radio buttons, and a dropdown menu.

SOLUTION:

```
import 'package:flutter/material.dart';
void main()
{
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      title: 'Form Example',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: FormPage(),
    );
  }
}
class FormPage extends StatefulWidget {
  const FormPage({super.key});
  @override
  State<FormPage> createState() => _FormPageState();
}

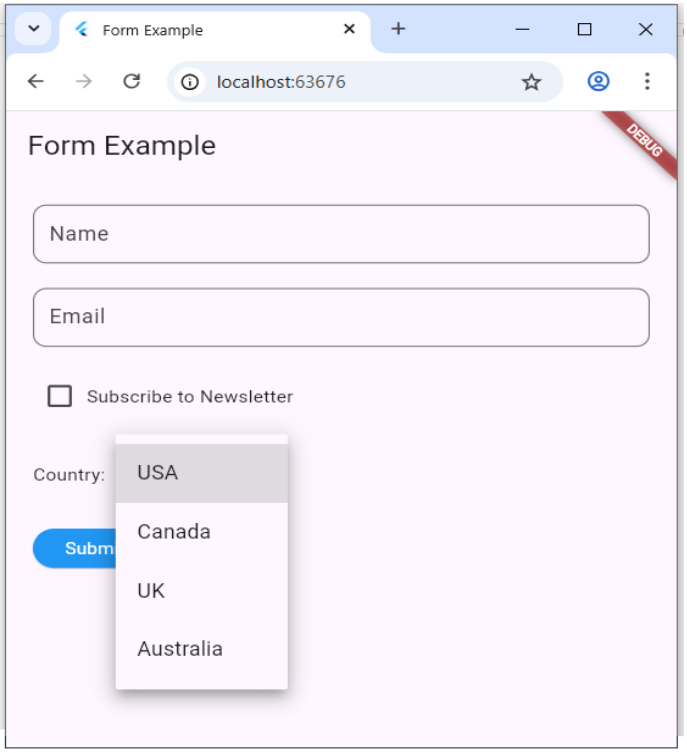
class _FormPageState extends State<FormPage>
{
  final _formKey = GlobalKey<FormState>();
  String ? _name;
  String ? _email;
  bool ? _subscribeToNewsletter = false;
  String ? _selectedCountry = 'USA';
  @override
  Widget build(BuildContext context)
  {
    return Scaffold(
      appBar: AppBar(
        title: Text('Form Example'),
```

```
),
body: Padding(
  padding: EdgeInsets.all(20.0),
  child: Form(
    key: _formKey,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        TextFormField(
          decoration: InputDecoration(
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(10.0),
            ),
            labelText: 'Name'),
          onSaved: (value) { _name = value;  },
        ),
        SizedBox(height: 20),
        TextFormField(
          decoration: InputDecoration(
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(10.0),
            ),
            labelText: 'Email'
          ),
          onSaved: (value) { _email = value; },
        ),
        SizedBox(height: 20),
        Row(
          children: <Widget>[
            Checkbox(
              value: _subscribeToNewsletter,
              onChanged: (value) {
                setState(() { _subscribeToNewsletter = value; }
                );
                },
            ),
            Text('Subscribe to Newsletter'),
          ],
        ),
        SizedBox(height: 20),
        Row(
          children: <Widget>[
            Text('Country: '),
            SizedBox(width: 20),
            DropdownButton<String>(
              value: _selectedCountry,
              onChanged: (value) { setState(() {
                _selectedCountry = value; });
                },
              items: <String>['USA', 'Canada', 'UK', 'Australia']
                  .map<DropdownMenuItem<String>>((String value)
              { return DropdownMenuItem<String>(
                value: value,
```

```
                child: Text(value),
              );
              }).toList(),
            ),
          ],
        ),
        SizedBox(height: 20),
        ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blue,
            foregroundColor: Colors.white,
          ),
            onPressed: () {
            _formKey.currentState!.save();
            // Submit the form data
            print('Name: $_name');
            print('Email: $_email');
            print('Subscribe to Newsletter: $_subscribeToNewsletter');
            print('Country: $_selectedCountry');
            },

          child: Text('Submit'),
        ),
      ],
    ),
  ),
 );
 }
}
```

OUTPUT:



Experiment - 7(b)

AIM: Implement form validation and error handling.

SOLUTION:

```
import 'package:flutter/material.dart';
void main()
{
 runApp(MyApp());
}
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   home: Scaffold(
    appBar: AppBar(
     title: Text('Form Example'),
    ),
    body: SingleChildScrollView(
     padding: EdgeInsets.all(16),
     child: FormWidget(),
    ),
   ),
  );
 }
}
class FormWidget extends StatefulWidget
{
 const FormWidget({super.key});
 @override
 State<FormWidget> createState() => _FormWidgetState();
}

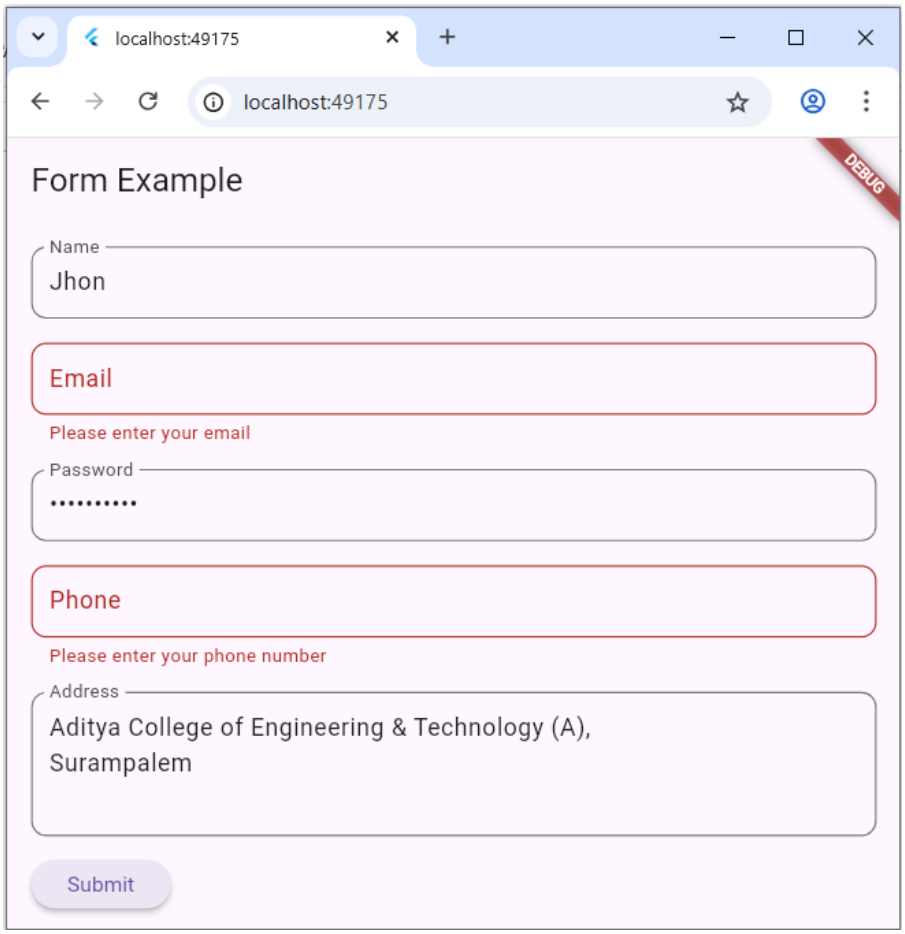
class _FormWidgetState extends State<FormWidget>
{
 final _formKey = GlobalKey<FormState>();
 String ? _name;
 String ? _email;
 String ? _password;
 String ? _phone;
 String ? _address;
 @override
 Widget build(BuildContext context)
 {
  return Form(
   key: _formKey,
   child: Column(
```

```
crossAxisAlignment: CrossAxisAlignment.start,
children: <Widget>[
 TextFormField(
  decoration: InputDecoration(
     border: OutlineInputBorder(
       borderRadius: BorderRadius.circular(10.0),
     ),
     labelText: 'Name'),
  validator: (value) {
   if (value!.isEmpty) {
    return 'Please enter your name';
   }
   return null;
   },
  onSaved: (value) => _name = value,
 ),
 SizedBox(height: 16),
 TextFormField(
  decoration: InputDecoration(
     border: OutlineInputBorder(
       borderRadius: BorderRadius.circular(10.0),
     ),
     labelText: 'Email'
  ),
  keyboardType: TextInputType.emailAddress,
  validator: (value)
  {
   if (value!.isEmpty) {
    return 'Please enter your email';
   }
   // Add more complex email validation logic if needed
   return null;
  },
  onSaved: (value) => _email = value,
 ),
 SizedBox(height: 16),
 TextFormField(
  decoration: InputDecoration(
     border: OutlineInputBorder(
       borderRadius: BorderRadius.circular(10.0),
     ),
     labelText: 'Password'
  ),
  obscureText: true,
  validator: (value)
  {
```

```
    if (value!.isEmpty) {
      return 'Please enter a password';
    }
    // Add more complex password validation logic if needed
    return null;
    },
  onSaved: (value) => _password = value,
),
SizedBox(height: 16),
TextFormField(
  decoration: InputDecoration(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10.0),
      ),
      labelText: 'Phone'),
  keyboardType: TextInputType.phone,
  validator: (value)
  {
    if (value!.isEmpty) {
      return 'Please enter your phone number';
    }
    // Add more complex phone number validation logic if needed
    return null;
    },
  onSaved: (value) => _phone = value,
),
SizedBox(height: 16),
TextFormField(
  decoration: InputDecoration(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10.0),
      ),
      labelText: 'Address'),
  maxLines: 3,
  validator: (value)
  {
    if (value!.isEmpty)
    {
      return 'Please enter your address';
    }
    return null;
    },
  onSaved: (value) => _address = value,
),
SizedBox(height: 16),
ElevatedButton(
```

```
            onPressed: _submitForm,
            child: Text('Submit'),
      ),
    ],
),
);
}
void _submitForm() {
  if (_formKey.currentState!.validate()) {
    _formKey.currentState!.save();
    // Perform form submission with the saved form data
    print('Form submitted:');
    print('Name: $_name');
    print('Email: $_email');
    print('Password: $_password');
    print('Phone: $_phone');
    print('Address: $_address');
    }
  }
}
```

OUTPUT:



Experiment - 8

AIM:
a) Add animations to UI elements using flutter's animation framework.
b) Experiment with different types of animations like fade, slide, etc.

DESCRIPTION:
Flutter's animation framework offers both implicit and explicit animations to enhance UI elements.

- **Implicit animations:** are suitable for simple property changes and when you prioritize ease of use. Examples: AnimatedContainer, AnimatedOpacity, AnimatedPositioned, AnimatedCrossFade).
- **Explicit animations:** are necessary for complex, custom animations, sequential animations, or when you need precise control over the animation's behaviour. Explicit animations are also prebuilt animation effects, but require an Animation object in order to work.
  Examples: SizeTransition, ScaleTransition or PositionedTransition.

Key Components:

- **Animation**: is a class that represents a running or stopped animation, and is composed of a value representing the target value the animation is running to, and the status, which represents the current value the animation is displaying on screen at any given time. It is a subclass of ***Listenable***, and notifies its listeners when the status changes while the animation is running.
- **AnimationController:** Manages the animation's duration, playback (forward, reverse, repeat), and status. Requires a *TickerProviderStateMixin* in the StatefulWidget.
- **Tween:** Defines the range of values an animated property will transition between (e.g., Tween<double>(begin: 0.0, end: 1.0)).
- **CurvedAnimation:** Applies a non-linear curve to the animation's progress, creating different easing effects (e.g., Curves.easeOut).
- **AnimatedBuilder** or **AnimatedWidget:** Rebuilds the UI based on the current animation value without rebuilding the entire widget tree, optimizing performance.

Experiment - 8(a)

AIM: Add animations to UI elements using flutter's animation framework.

SOLUTION:

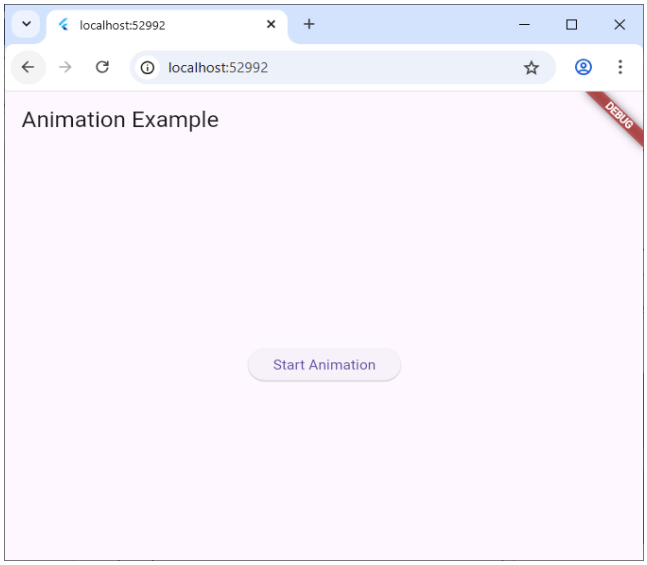
```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Animation Example'),
        ),
        body: AnimationWidget(),
      ),
    );
  }
```

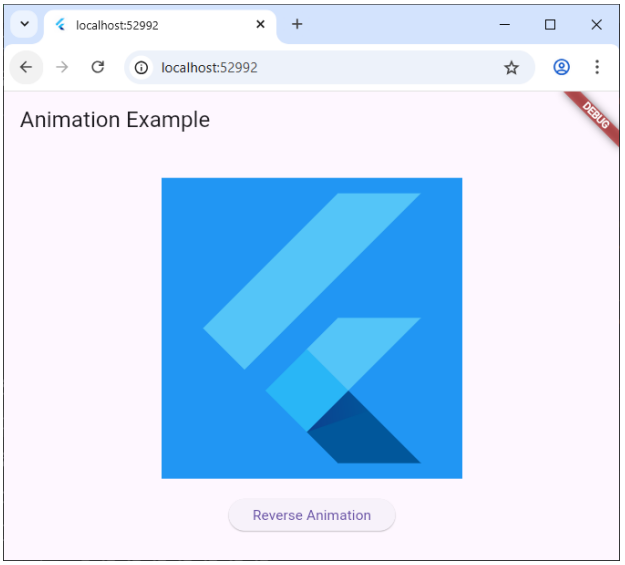
```
}
class AnimationWidget extends StatefulWidget
{
  @override
  _AnimationWidgetState createState() => _AnimationWidgetState();
}

class _AnimationWidgetState extends State<AnimationWidget>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;
  @override
  void initState()
  {
    super.initState();
    _controller = AnimationController(
      duration: Duration(seconds: 1),
      vsync: this,
    );
    _animation = Tween<double>(begin: 0, end: 300).animate(_controller)
      ..addListener(()
      {
        setState(() {}); // Trigger rebuild when animation value changes
      });
}
@override
Widget build(BuildContext context)
{
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Container(
          width:  _animation.value,
          height: _animation.value,
          color: Colors.blue,
          child: FlutterLogo(size: 100),
        ),
        SizedBox(height: 20),
        ElevatedButton(
          onPressed: ()
          {
            if (_controller.status == AnimationStatus.completed)
            { // Restart animation
              _controller.reverse();
            }
            else
            { // Start animation
              _controller.forward();
            }
          },
          child: Text(
            _controller.status == AnimationStatus.completed
```

```
            ? 'Reverse Animation'
            : 'Start Animation',
          ),
        ),
      ],
    ),
  );
}
@override
void dispose()
{ // Clean up controller when widget is disposed
  _controller.dispose();
  super.dispose();
}
}
```

Output:





Experiment - 8(b)

AIM: Experiment with different types of animations like fade, slide, etc.

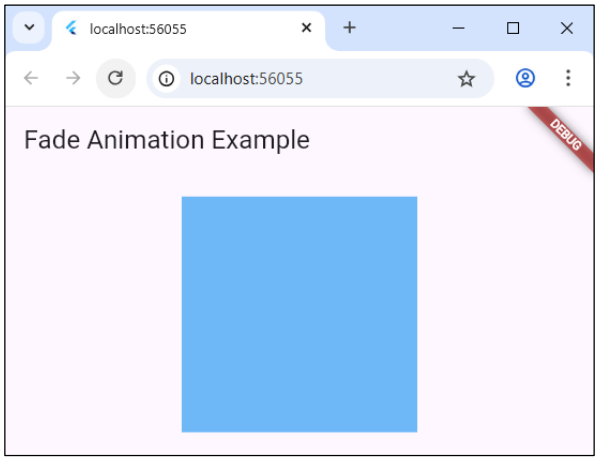
SOLUTION:

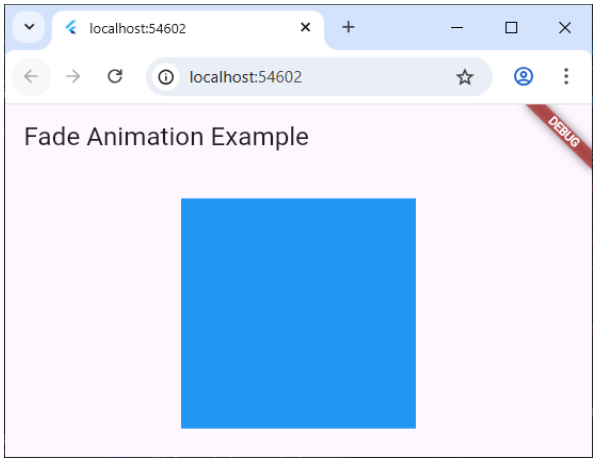
Fade Animation:

```
import 'package:flutter/material.dart';
void main()
{
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Fade Animation Example'),
        ),
        body: FadeAnimation(),
      ),
    );
  }
}
class FadeAnimation extends StatefulWidget
{
  @override
  _FadeAnimationState createState() => _FadeAnimationState();
}
class _FadeAnimationState extends State<FadeAnimation>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;
  @override
  void initState()
  {
    super.initState();
    _controller = AnimationController(
      duration: Duration(seconds: 2),
      vsync: this,
    );
    _animation = Tween<double>( begin: 0, end: 2).animate(_controller);
    _controller.forward(); // Start animation automatically
  }
  @override
  Widget build(BuildContext context)
  {
    return
      Center(
        child: FadeTransition(
          opacity: _animation,
```

```
          child: Container(
            width: 200,
            height: 200,
            color: Colors.blue,
          ),
        ),
      );
  }
  @override
  void dispose()
  { // Clean up controller when widget is disposed
    _controller.dispose();
    super.dispose();
  }
}
```

Output:



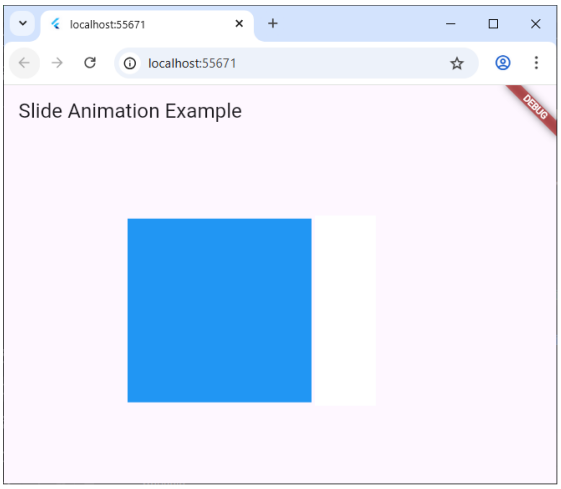


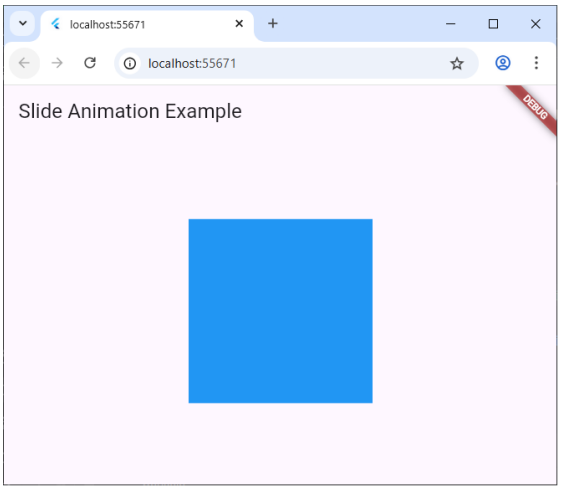
Slide Animation:

```
import 'package:flutter/material.dart';
void main()
{
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Slide Animation Example'),
        ),
        body: SlideAnimation(),
      ),
    );
  }
}
class SlideAnimation extends StatefulWidget
{
  @override
  _SlideAnimationState createState() => _SlideAnimationState();
}
class _SlideAnimationState extends State<SlideAnimation>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<Offset> _animation;
  @override
  void initState()
  {
    super.initState();
    _controller = AnimationController(
      duration: Duration(seconds: 2),
      vsync: this,
    );
    _animation = Tween<Offset>(
        begin: Offset(-1.0, 0.0),
        end: Offset(0.0, 0.0)
    ).animate(_controller);
    _controller.forward(); // Start animation automatically
  }
  @override
  Widget build(BuildContext context)
  {
    return
      Center(
        child: SlideTransition(
          position: _animation,
          child: Container(
            width: 200,
```

```
            height: 200,
            color: Colors.blue,
          ),
        ),
      );
  }
  @override
  void dispose()
  { // Clean up controller when widget is disposed
    _controller.dispose();
    super.dispose();
  }
}
```

Output:



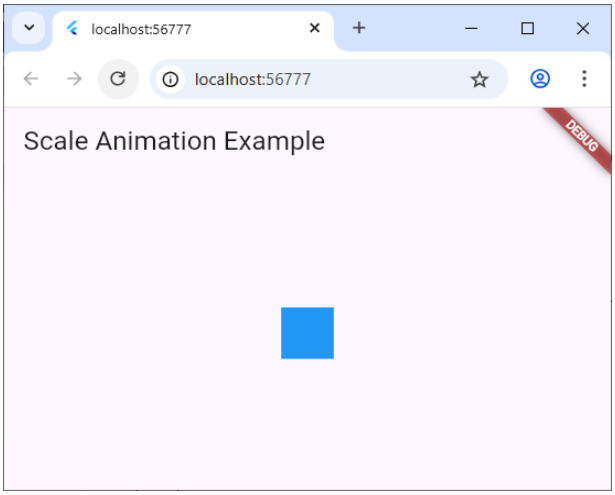


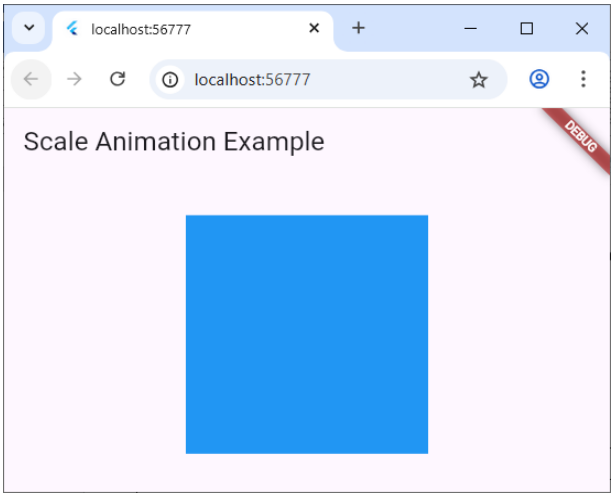
Scale Animation:

```
import 'package:flutter/material.dart';
void main()
{
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Scale Animation Example'),
        ),
        body: ScaleAnimation(),
      ),
    );
  }
}
class ScaleAnimation extends StatefulWidget
{
  @override
  _ScaleAnimationState createState() => _ScaleAnimationState();
}
class _ScaleAnimationState extends State<ScaleAnimation>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;
  @override
  void initState()
  {
    super.initState();
    _controller = AnimationController(
      duration: Duration(seconds: 2),
      vsync: this,
    );
    _animation = Tween<double>( begin: 0.0, end: 1.0).animate(_controller);
    _controller.forward(); // Start animation automatically
  }
  @override
  Widget build(BuildContext context)
  {
    return Center(
      child: ScaleTransition(
        scale: _animation,
        child: Container(
          width: 200,
          height: 200,
          color: Colors.blue,
        ),
      ),
```

```
    );
  }
  @override
  void dispose()
  { // Clean up controller when widget is disposed
    _controller.dispose();
    super.dispose();
  }
}
```

Output:





Experiment - 9

AIM:
a) Fetch data from REST API.
b) Display the fetched data in a meaningful way in the UI.

DESCRIPTION:
A REST API in Flutter refers to the integration and utilization of a Representational State Transfer (REST) Application Programming Interface (API) within a Flutter application. APIs are often used to fetch data from a server or a database. HTTP requests, specifically GET requests, are commonly employed to retrieve data from APIs. When a Flutter app sends a GET request to an API endpoint, it receives a response containing the requested data, typically in JSON format.

Flutter applications use the http package (or other third-party packages like Dio) to make HTTP requests to a REST API. The process typically involves:

- **Making HTTP Requests:**
  Using methods like http.get(), http.post(), http.put(), or http.delete() to send requests to specific API endpoints.
- **Handling Responses:**
  The server responds with data, usually in JSON format. Flutter applications parse this JSON data into Dart objects.
- **Updating UI:**
  The parsed data is then used to update the user interface of the Flutter application, displaying information fetched from the server or reflecting changes made through API calls.

Fetch Data from a REST API in Flutter:
STEP-1: Include the http package in your pubspec.yaml file

```
dependencies:
      flutter:
        sdk: flutter
      http: ^1.2.1 # Use the latest version
```

STEP-2: run `flutter pub get` in your terminal. Import the http package.

STEP-3: Create a Dart file with http packages imported and an asynchronous function to fetch data that makes the HTTP GET request and handles the response.

Experiment – 9(a)

AIM: Fetch data from REST API.

SOLUTION:

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
void main()
{
 runApp(MyApp());
}
class MyApp extends StatelessWidget
{
 @override
 Widget build(BuildContext context)
 {
```

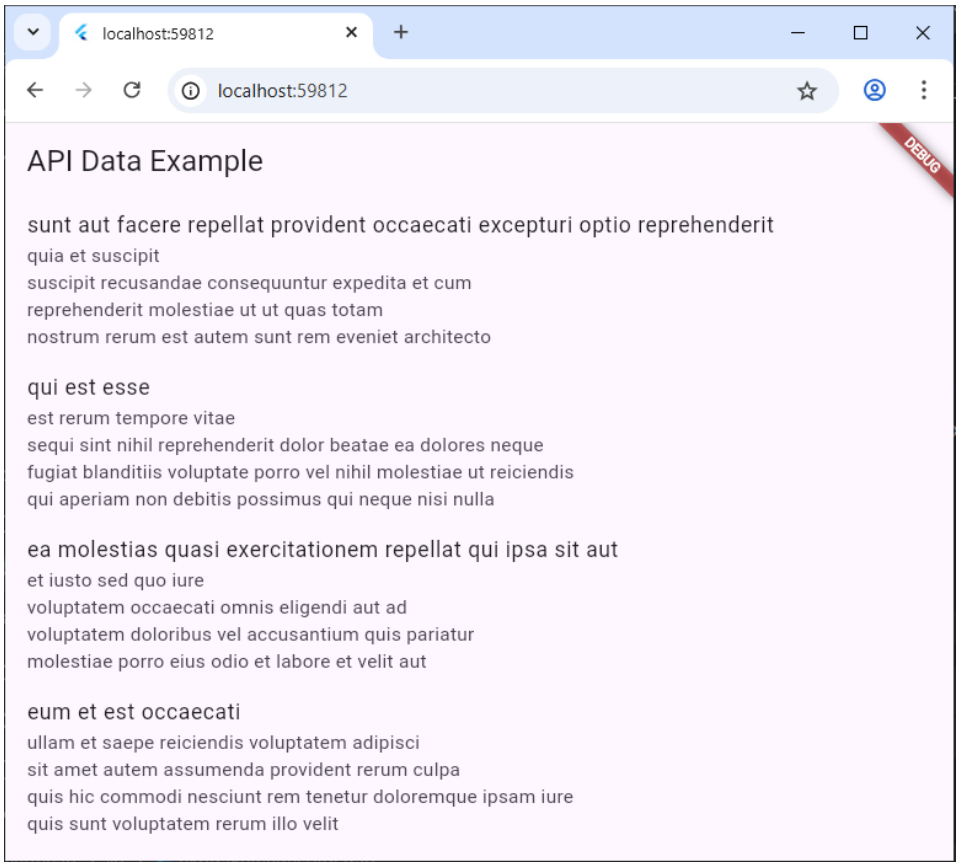
```
    return MaterialApp(
      home: HomePage(),
    );
  }
}
class HomePage extends StatefulWidget
{
  @override
  _HomePageState createState() => _HomePageState();
}
class _HomePageState extends State<HomePage> {
  List<dynamic> _data = [];
  bool _isloading = false;
  @override
  void initState() {
    super.initState();
    _fetchDataFromApi();
  }

  Future<void> _fetchDataFromApi() async
  {
    setState(() {
      _isloading = true;
    });
    final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    if (response.statusCode == 200) { // If the server returns a 200 OK response, parse the JSON.
      setState(() {
        _data = json.decode(response.body);
        _isloading = false;
      }
      );
    }
    else { // If the server did not return a 200 OK response,
      // throw an exception.
      throw Exception('Failed to load data');
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('API Data Example'),
      ),
      body: ListView.builder(
```

```
        itemCount: _data.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(_data[index]['title']),
            subtitle: Text(_data[index]['body']),
          );
        },
      ),
    );
  }
}
```

Output:



Experiment – 9(b)

AIM: Display the fetched data in a meaningful way in the UI.

DESCRIPTION:

To display the fetched data in a meaningful way in the UI, we can use a more structured layout rather than just displaying the data in a list. We'll create a custom widget to represent each post fetched from the API, and display them in a scrollable list.

We've added a loading indicator (CircularProgressIndicator) to indicate when data is being fetched. The fetched data is displayed as a list of PostCard widgets, each representing a post from the API. The PostCard widget displays the title and body of each post in a structured manner using a Card layout.

SOLUTION:

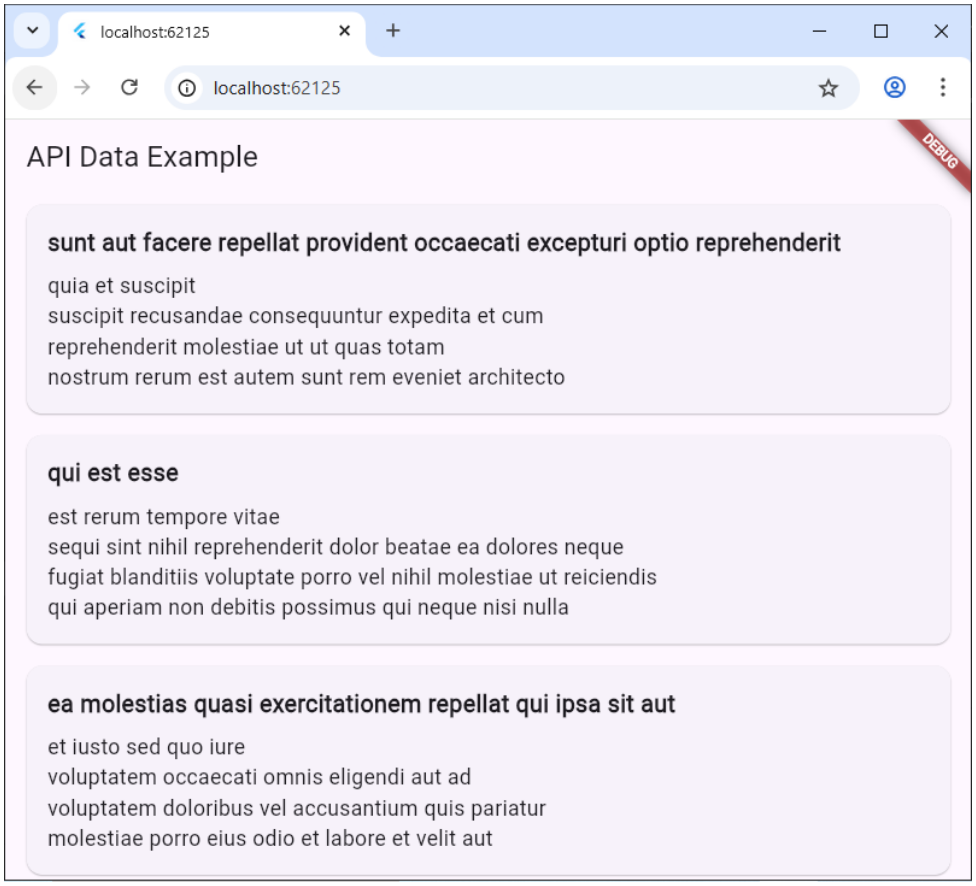
```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
void main()
{
  runApp(MyApp());
}
class MyApp extends StatelessWidget
{
  @override
  Widget build(BuildContext context)
  {
    return MaterialApp(
      home: HomePage(),
    );
  }
}
class HomePage extends StatefulWidget
{
  @override
  _HomePageState createState() => _HomePageState();
}
class _HomePageState extends State<HomePage> {
  List<dynamic> _data = [];
  bool _isloading = false;
  @override
  void initState() {
    super.initState();
    _fetchDataFromApi();
  }

  Future<void> _fetchDataFromApi() async
  {
    setState(() {
      _isloading = true;
    });
    final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    if (response.statusCode == 200)
    { // If the server returns a 200 OK response, parse the JSON.
```

```
      setState(() {
        _data = json.decode(response.body);
        _isloading = false;
      }
      );
    }
    else { // If the server did not return a 200 OK response,
      // throw an exception.
      throw Exception('Failed to load data');
    }
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('API Data Example'),
      ),
      body: _isloading ? Center(
        child: CircularProgressIndicator(),
      )
      : ListView.builder(
        itemCount: _data.length,
        itemBuilder: (context, index) {
          return PostCard(
            title: _data[index]['title'],
            body:  _data[index]['body'],
          );
        },
      ),
    );
  }
}
class PostCard extends StatelessWidget {
  final String title;
  final String body;
  const PostCard({
    Key? key,
    required this.title,
    required this.body,
  }) : super(key: key);
  @override
  Widget build(BuildContext context)
  {
    return Card(
      margin: EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      child: Padding(
        padding: EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text( title,
              style: TextStyle(
                fontSize: 18,
```

```
                fontWeight: FontWeight.bold
              ),
            ),
            SizedBox(height: 8),
            Text(body,
              style: TextStyle(fontSize: 16
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:



Experiment – 10

AIM:
a) Write unit tests for UI components.
b) Use flutter's debugging tools to identify and fix issues.

Experiment – 10(a)

AIM: Write unit tests for UI components.

DESCRIPTION:
In Flutter, testing UI components is primarily achieved through Widget Tests, not traditional unit tests. Widget tests are specifically designed to verify the behavior and appearance of UI components (widgets).

Here's how to write widget tests for UI components in Flutter:

- Add the **flutter_test** dependency: This package is included by default in new Flutter projects.

```
dev_dependencies:
  flutter_test:
    sdk: flutter
```

- run "flutter pub get" to install the flutter_test package. (Not required if flutter_test dependency is already available)
- **Create a test file:** Create a new Dart file in your **test** folder, typically named [widget_name]_test.dart
- **Import necessary packages:**

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
// Import your widget file
import 'package:your_app_name/widgets/my_widget.dart';
```

- **Write your widget test:** Use the **testWidgets** function to define your test cases. This functionprovides a **WidgetTester** object that allows you to interact with and inspect your widget tree.
  - ✓ **Key elements of widget testing:**
    - **tester.pumpWidget(widget):** Renders the provided widget in an isolated test environment.
    - **find methods**: Used to locate widgets in the widget tree (e.g., find.text(), find.byType(), find.byKey()).
    - **expect and matchers**: Used to assert conditions (e.g., findsOneWidget, findsNWidgets, findsNothing).
    - **tester.tap(finder)**: Simulates a tap on a widget.
    - **tester.pump()**: Rebuilds the widget tree, reflecting any state changes after user interactions or animations.
- **Run the tests:** You can run your tests from your IDE (e.g., VS Code, IntelliJ) or by executing flutter test in your terminal.

SOLUTION:

```
// a test file in the test directory
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
// Import your widget file
import 'package:flutterlab/exp2cButtonCounter.dart';
void main()
{
 testWidgets('Counter starts at 0',
       (WidgetTester tester) async {
   await tester.pumpWidget(
      const MaterialApp(home: MyHomePage()));

   // Find the Text widget by its content
   final textFinder1 = find.text('Counter Value:');

   // Verify that the Text widget is found
   expect(textFinder1, findsOneWidget);

   // Find the Text widget by its content
   final textFinder2 = find.text('0');

   // Verify that the Text widget is found
   expect(textFinder2, findsOneWidget);
 }
 );
 testWidgets('Counter increments when button is pressed',
       (WidgetTester tester) async {
      await tester.pumpWidget(
       const MaterialApp(home: MyHomePage()));

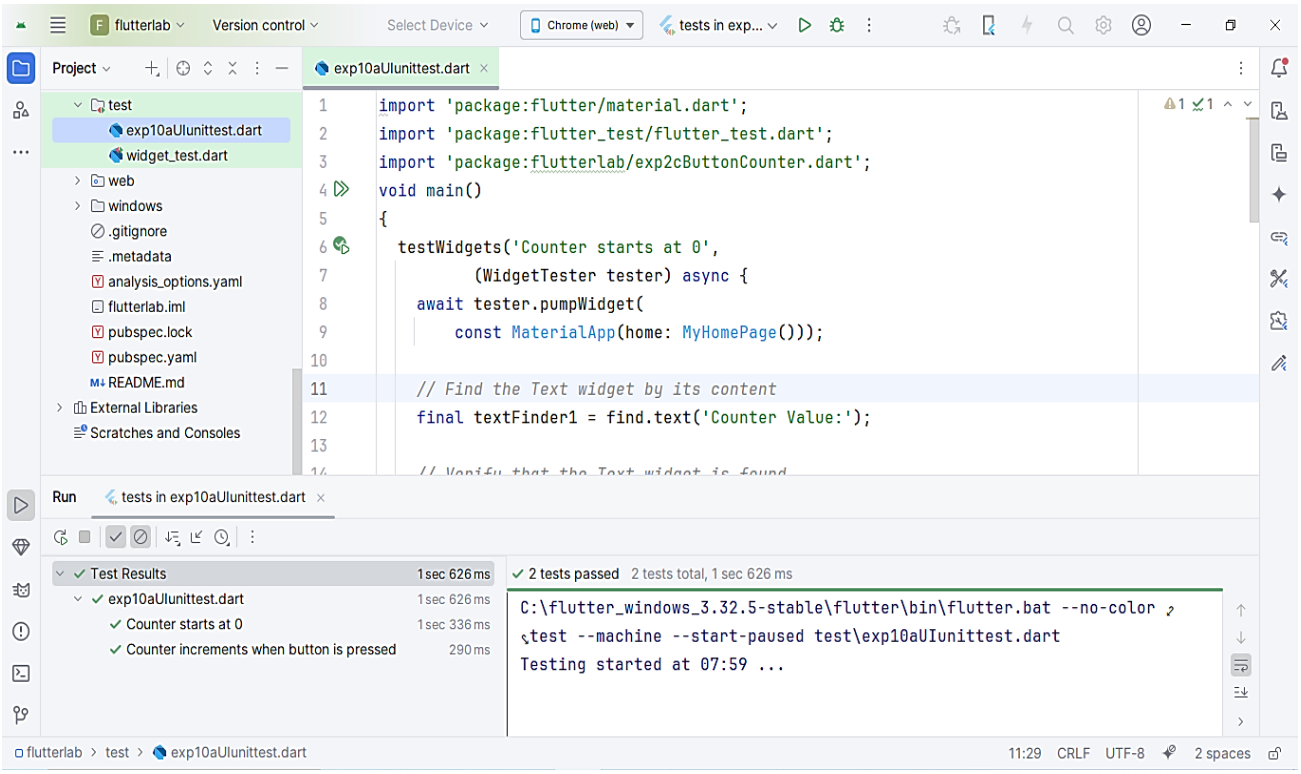
      // Find the button or tappable area
      final buttonFinder = find.text('Increment Counter');
      expect(buttonFinder, findsOneWidget);

      // Simulate a tap
```

```
    await tester.tap(buttonFinder);
    await tester.pump(); // Rebuild the widget tree after the tap

    // Verify the expected change (e.g., text changes, state updates)
    expect(find.text('0'), findsNothing);
    expect(find.text('1'), findsOneWidget);
  });
}
```

Output:



Experiment – 10(b)

AIM: Use flutter's debugging tools to identify and fix issues.

DESCRIPTION:
Flutter offers a comprehensive suite of debugging tools to help identify and fix issues in your applications. These tools are primarily integrated into your IDE (like VS Code or Android Studio) and accessible through Dart DevTools.

1. Dart DevTools:
Dart DevTools is a web-based suite of tools for debugging and profiling Flutter and Dart applications. It can be launched from your IDE during a debug session. Key features include:

- **Widget Inspector:** Visually inspect the widget tree, understand layout, and diagnose rendering issues. You can select widgets in the running app and see their properties and position in the tree, or vice-versa.
- **Performance View:** Analyse application performance, including frame rendering times, CPU usage, and network activity. Identify bottlenecks and optimize for smoother UI.
- **Memory Profiler:** Detect and analyse memory leaks, track memory allocation, and optimize memory usage to prevent performance degradation.
- **CPU Profiler:** Profile CPU usage to identify performance bottlenecks in your code and optimize execution efficiency.
- **Network Profiler:** Monitor network requests and responses, analyze network performance, and optimize API calls.
- **Logging View:** View and filter logs from your application, helping to understand the flow of execution and identify errors.

2. IDE Debugging (VS Code/Android Studio):
Your IDE provides powerful built-in debugging capabilities:

- **Breakpoints:** Set breakpoints in your Dart code to pause execution at specific lines, allowing you to inspect variable values, call stacks, and the overall state of your application at that point.
- **Step-through Debugging:** Step through your code line by line, entering or stepping over functions, to meticulously follow the execution flow and understand how your program behaves.
- **Variable Inspection:** Examine the values of variables in scope at breakpoints, helping to pinpoint incorrect data or unexpected changes.
- **Debug Console:** View application logs and print statements, providing valuable insights into the program's execution.

3. Other Useful Tools and Techniques:

- **flutter doctor:** This command line tool checks your Flutter development environment for common issues like missing dependencies or incorrect SDK versions. Running it first can often resolve setup-related problems.
- **Print Statements and Logging:** Simple print() statements or more structured logging using the dart:developer library can help you track variable values, execution paths, and debug specific sections of your code.
- **Assertions:** Use assert() statements to validate assumptions in your code during development. These assertions are only active in debug mode and can help catch logical errors early.
- **Hot Reload:** While not strictly a debugging tool, Hot Reload is invaluable for quickly iterating on UI changes and seeing the effects of your code modifications without restarting the entire application, speeding up the debugging process.

SOLUTION:

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Counter App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0; // Initialize the counter value

  void _incrementCounter() {
    setState(() {
      // Increment the counter and update the UI
      _counter++;
    });
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter App'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'Counter Value:',
              style: TextStyle(fontSize: 20),
            ),
            Text(
              '$_counter', // Display the current counter value
```

```
            style: Theme.of(context).textTheme.headlineMedium,
          ),
          const SizedBox(height: 30), // Add some spacing
          ElevatedButton(
            onPressed: _incrementCounter, // Call the increment function on button press
            child: const Text('Increment Counter'),
          ),
        ],
      ),
    ),
  );
 }
}
```

Output:

